DENIZEN — Technical Dossier v1.8.0

Full narrative source render · commit 1cab7977 · 2026-09-16

DENIZEN — Technical Dossier

Version 1.8.0 · source of record · headline metrics re-measured at commit `1cab7977` (2026-09-16); subsystem deep-survey still anchored to `d25bc7a2` (2026-07-25) unless noted

This is the maintained, in-repo source for the DENIZEN Technical Dossier. The branded PDF is

rendered from this file plus `data/canonical/denizen-facts.json` — it is not authored separately.

Skin, tokens and the render chain are specified in BRANDED_DOSSIER_SYSTEM.md.

> **Why this file exists.** The v1.6.0 PDF lived only as a standalone export pinned to `ae3ddf9`.
>
> v1.7.0 moved the dossier into the repo. v1.8.0 catches the next product wave: **Buzz, Workbench,
>
> Seat an AI, Studio Board, CRM member scope / team viewer, and living-office NPC behavior** — none
>
> of which appear in the uploaded v1.6.0 PDF. See [Appendix E](#appendix-e--delta-from-v160) and
>
> [Appendix E2](#appendix-e2--delta-from-v170) plus [Appendix D](#appendix-d--verification-appendix).

Front matter

How to read this document

If you are a…	Read	What you get
Engineer / technical evaluator	Part I (all), Part III (all), Appendices C–D	How every subsystem works, how to run and maintain it, the module and route map, and a troubleshooting runbook
Investor / partner	Part I §1–2, Part II, Part IV	What the product is, who it is for, the wedge, the requirement and release plan, and third-party evidence of how it was built
Marketer / GTM	Part I §1, Part II §2–3, Part IV, Appendix F	The one-line story, personas and jobs-to-be-done, competitive framing, and ready-to-use positioning language
Operator on call	Part III, Appendix A, [Appendix G](#appendix-g--open-defect-register)	Install, ports, runbook, and the current defect register

The four parts

- **Part I — Technical & Architectural Breakdown.** What was built, why, and exactly how it works.

- **Part II — Product Requirements Document.** Problem, vision, personas, requirements, metrics, release plan, risks.

- **Part III — Technician & Maintenance Manual.** Prerequisites, install, config, topology, operations, runbook, schedule.

- **Part IV — Builder Profile & Signal.** An independent retrospective of how the system was built.

Provenance and honesty rules

Headline scale numbers (server LOC, module count, migrations, docs, test files, roster size) were re-measured at `1cab7977` on 2026-09-16. Deeper subsystem claims that are unchanged since July still cite the nine surveys at `d25bc7a2` (2026-07-25). Where a survey and an existing doc disagreed, **the source tree won and the disagreement is recorded** rather than quietly resolved.

Status tags are used throughout and mean exactly this:

Tag	Meaning
`[DONE]`	Implemented, and exercised by a test or a verified run
`[PARTIAL]`	Implemented but incomplete, degraded, or not reaching production
`[UI-ONLY]`	A working interface over mock data — no real backend behavior
`[PLANNED]`	Specified, not built
`[BROKEN]`	Implemented and currently failing; tracked in [Appendix G](#appendix-g--open-defect-register)

What this document does not claim.

- "Built" does not mean "applied to every production tenant." Production previously lagged at **47 of 51 migrations**; the repo now has **52** migrations — re-verify against the live NAS before quoting applied counts (§7 / §8 / Part III M6).

- Full suite pass/fail counts below that are still labeled July were **not** re-run for v1.8.0. Headline LOC / file counts were. Production is deployed by `infra/deploy-denizen-to-nas.py` and patched by `hotfix-*.py`, so **the running system is routinely ahead of both the last image and the last commit.**

- No revenue, fundraising, or customer-count figure appears here. Those are operator-supplied only.

- The ~82.5% prediction-accuracy figure carried by earlier versions remains **withdrawn pending re-derivation** — see §7.

- Version 1.8.0 is a working label for the September 2026 customer-surface wave. Release numbering is the operator's call.

Current state at a glance — v1.8.0

Metric	Value	Provenance
Pinned commit	**`1cab7977`** (master, 2026-09-16)	`git rev-parse`
Commits since v1.7.0 pin	**119** · `+536k / −8k` lines across 469 files	`git log` / `git diff --shortstat`

`server.js`	**13,929 lines**, raw `http.createServer`, no framework	measured	
`src/` modules	**394** `.js` files under `src/`	measured	
Total tracked JS	**257,563 LOC** across **886** tracked `.js`/`.mjs` files	`git ls-files \`	xargs wc -l`
Migrations	**52** ordered, additive-only	measured	
Agent roster	**18 live agents** (`src/npc-roster.js`)	measured	
Docs	**97** in `denizen/docs/`	measured	
Test files	**212** under `tests/`	measured	
Customer product objects	**Floor · Workbench · Buzz** (+ Seat an AI, Meridian CRM, Studio Board)	`landing.html`, shipped modules	
Tests (July baseline)	**1,432 pass / 7 fail** of 1,439 — suite exits 1	live run at `d25bc7a2` — re-run before citing	
Jelly / MCP / Discord (July)	**29 skills · 38 MCP tools · 130 Discord commands**	July survey — re-enumerate before citing	

Known open items. Production previously ran 47 of 51 migrations; the repo now has **52** — confirm what is applied on the NAS before calling the spine "live" (§7). The desktop app was unbuildable as of the July survey (§16). Seven tests failed at the July baseline — re-run before claiming green. Auth coverage gaps and backup gaps remain in [Appendix G](#appendix-g--open-defect-register). Customer surfaces shipped since July (Buzz / Studio Board / Seat an AI) are **[DONE]** in the office chrome; mobile Buzz had a touch-ring regress that is fixed on `cursor/buzz-mobile-fix-f4a2` (PR #73) pending merge.

PART I — Technical & Architectural Breakdown

01 · What Denizen is

Denizen is a real-time visual observability layer for AI agents. Instead of reading terminal logs and JSON traces to understand what a fleet of autonomous agents is doing, you watch a living pixel-art office in which every character is a real LLM-backed agent doing a real job.

Agents walk to desks, hold meetings, write reports, file bugs, review code, run web research, watch the market, and investigate security threats — and each action renders as character behavior you can read at a glance. The insight is that agent workflows are far easier to debug, monitor and trust when you can see the *shape* of the work: who is idle, who is blocked, what is chained to what, where the bottleneck is.

> **The product, in one sentence.** Denizen is a multi-tenant AI-agent operating system that renders

> autonomous agents as a live pixel-art office so their work is observable; each agent is a real LLM
>
> with a soul file, persistent memory, real tools, and human-approval gates, and the whole system logs
>
> facts → outcomes → predictions so it gets measurably smarter over time.

Since v1.6.0 the product has grown a second surface that the earlier dossier did not describe at all:

a Discord-native version of the same idea (§11) — a bot, a walkable pixel "Spaces" map, and a per-guild paid tier. Measured by commit volume it was the single largest area of work in the period, and it is the first surface with real billing attached.

Three lenses on the same system

Lens	What Denizen is
For an observer	A pixel-art office where colored status dots and agent positions show live state. Press Tab for a terminal-style work feed; press Enter to talk to any agent as the CEO.
For an engineer	A 13,537-line Node.js server with **no framework** — raw `http` + `ws` — orchestrating 334 runtime modules, a Jelly skill runtime, a Supabase intelligence spine, two WebSocket channels, two SSE streams, 331 routes, and pluggable local + cloud LLM providers.
For a business	A multi-tenant agent operating layer. Each durable role is one agent; the roster remaps to any company by editing soul files and the org chart. Two monetized surfaces exist today: the Discord guild tier (Stripe, live) and the CRM client portal (Stripe Checkout).

02 · The core idea & why NPCs

When you run AI agents in production you usually end up staring at logs, JSON, or trace viewers. You can tell what happened, but not the shape of the work. Denizen renders that telemetry as a place.

One agent per durable role. A "role" is anything with its own decision context, success criteria, files it cares about, and escalation paths. The rule in the codebase is explicit: if you can describe a job in two sentences, it is an NPC; the moment you write "and also does X," X should be its own NPC.

Identity is a file, not code. Each agent reads itself into existence from `SOUL.md` at startup. Edit the file, change the agent — no code change, fully version-controlled, model-agnostic. This is what makes the roster a template rather than a product, and it is the mechanism behind every remapping in Part II §2.

> **The thesis, in the project's own framing.** Denizen's roster is not a fixed product — it is a
>
> template for how to structure an AI-powered team so the work is observable. Fork it, swap the soul
>
> files, change the org chart, and you have a visual operating layer for whatever your business does.

03 · System architecture & the two decision loops

Denizen is a browser client talking to a Node.js server, which talks to LLM providers, a

Postgres/Supabase database, the Jelly skill runtime, an optional Linux security sidecar, and a distributed compute mesh.

High-level topology



No framework. There is no Express, no router, no middleware stack. Routing is one `http.createServer` callback (`server.js:3695`) containing a ~7,400-line sequential first-match-wins `if (urlPath === …)` chain plus 21 regex `.match()` sites. Runtime dependencies total **seven**: `ws`, `redis`, `playwright`, `googleapis`, `jimp`, `@nut-tree-fork/nut-js`, `@tabler/icons-webfont`.

Two WebSocket channels, not four. `server.on('upgrade')` (`server.js:12965`) routes exactly `/security-ws` (`:11131`) and `/agent-ws` (`:11172`, 503s at capacity, `MAX_WS_CLIENTS` = 50, or 30 in public-demo mode). Everything else is destroyed unless the optional OpenClaw reverse proxy is enabled, which it is not by default. The `/gateway-ws` and `/mesh-ws` channels described by earlier versions **no longer exist**; that traffic moved to HTTP plus **two SSE streams**: `GET /api/crm/workspace/stream` and `GET /api/compute/inference/stream`.

The request prologue. Every request walks this fixed order before reaching any route branch: `safeDecodePath` → **`securityMonitor.checkHttpRequest` on every request** → `/invite//join` redirects → a hoisted visitor-auth fast path (commented "must stay fast (login modal)") → `_handleVisitorAuthRoutes` → prefix mounts (`/api/office/`, `/api/crm/`, `/api/n8n/`, `/api/compute/` + `/api/desktop/`) → MCP OAuth front door → MCP gateways and `/api/npc-sync/*` →

optional OpenClaw proxy → ~7,000 lines of inline branches → path aliasing and static fallback.

Auth is header-based, not cookies. `_verifyVisitorSession` (`:1777`) reads `X-Session-Id` and `X-Session-Token`, plus `X-Access-Grant`, `X-Denizen-Ceo-Grant`, `X-Org-Slug`, `X-Email`. GET-with-header fallbacks appear throughout, because "Cloudflare tunnel may not forward POST bodies to origin" (`:3724`).

Org scoping is two-layer and not uniform. Per-session orgs resolve from `session.org_id` via team code. But a **process-global single-tenant fallback** `denizenDb.orgId` (default slug `zion`, `src/denizen-db.js:40`) is used directly by `/api/federal/*` and `/api/intelligence/*`. The resolution chain `session?.org_id || denizenDb.orgId || getDefaultOrgId() || '__default__'` appears at `:983` and `:3603`. CRM is the strictest surface — it returns 409 when a session has no org.

Rate limiting is near-absent. `[PARTIAL]` There is no global limiter. Four ad-hoc 429s exist, all per-NPC placement budgets, plus IP throttling inside CRM intake and a 120/token/hour limit on the per-NPC MCP gateway. **The team MCP gateway has no rate limit at all.**

⚠ The auth surface

237 of 372 dispatch sites (64%) carry no detectable auth check. Many are deliberately public — the CRM intake lead form, health, OAuth callbacks. But the following were verified open by reading the handler bodies, not inferred:

Route	File	Why it matters
`POST /api/assign-directive`	`server.js:5228`	Makes any NPC do work
`POST /api/grant-permission`	`:5136`	Grants agent permissions
`POST /api/soul-proposal/apply`	`:4625`	**Mutates NPC persona**
`POST /api/market/paper/order`, `/paper/reset`	market group	17/17 market routes open
all 14 telemetry/admin routes	`:6367` etc.	Full LLM telemetry, costs, decisions
`GET /api/world-state`	`:6991`	Complete office state
`GET /api/federal/pipeline-summary`	`:7740`	See §7 — also cross-org
`/api/tts`, `/api/stt`, 4 n8n routes, `POST /api/job-apply`	—	Provider-cost surfaces

This is on a host reachable through a Cloudflare tunnel. `soul-proposal/apply` and `assign-directive` are the two most autonomy-relevant writes in the system, which puts them in direct tension with the approval-gate thesis in §6. **No in-repo evidence of upstream enforcement was found.** If the tunnel or a proxy enforces auth, that fact belongs in this document; if not, this is the top item in [Appendix G](#appendix-g--open-defect-register).

Two adjacent notes: `ROOT = __dirname` (`:834`) makes the application directory itself the static docroot — the `path.relative` guard blocks `..` and raises a `high`-severity threat event, but

anything *inside* `denizen/` is fetchable, including `.env` if it sits there. And `src/route-registry.js` is a parallel registry that re-declares `/api/market/trade`, `/api/job-crm/maintenance` and `/api/competency-map`, carrying its own in-code warning: "inline handlers in this file remain canonical … Do not double-register without audit."

Loop 1 — the per-agent think loop (bottom-up)

The loop is **browser-scheduled, server-executed**: the scheduler is `src/agent-office-manager.js:1952` `_npcThinkLoop()`, the WS entry is `server.js:11314`, `src/npc-think-runner.js:136` is a wake/debounce wrapper (not the loop), and cognition happens in `src/npc-brains.js:2523` `think()`. There are **24 stages** and **10 distinct triggers**; notably there is **no player-proximity trigger and no chat trigger** — player position is prompt context only, and active chat *suppresses* thinking with a 45 s deferral.

Cadence (authoritative: `src/prompt-tuning.js:49-56`) — **120,000–200,000 ms**, i.e. one think every 2–3.3 minutes per agent, 150–240 s in economy mode, with a 10 s stagger, an 8 s global gap, and a 2.5× multiplier on mobile. The cofounder loop is 45 s. ⚠ Earlier versions of this document said 45–75 s; that is wrong.

⚠ **Cadence bug.** The browser fetches `/api/office-config` (`server.js:9988`), but its hardcoded fallback disagrees with the server: `agent-office-manager.js:318-321` uses `thinkMinMs ?? 360000` and `thinkMaxMs ?? 600000` — 6 and 10 minutes against the configured 2 minutes and 200 s. **If that fetch fails, the whole office thinks about 3× slower than configured, silently.**

Each cycle:

1. `NpcBrainManager.think()` assembles a 22-section context block: `SOUL.md`, the last ~1,200 chars of `MEMORY.md`, ~10 recent conversations, the persistent goal, the day's plan, a theory of mind of the four nearest agents, recent office events, the shared task board, org hierarchy, mood, nearby furniture.
2. One chat/completions call goes to the agent's provider — `max_tokens ≈ 400`, temperature 0.95.
3. The model returns one JSON decision (contract in §4).
4. The server saves memory (tagging `[OUTCOME:*] [TOPIC:*] [SKILL:*]`), updates goal progress, records the relationship, and broadcasts an office event for meetings, ships and blockers.
5. The decision goes to the client, which executes the matching action primitive.

> **Why one GPU stays sane.** All LLM calls serialize through a single queue so a local GPU runs one inference at a time. `LM_STUDIO_PARALLEL` clamps the interactive inflight cap to 1–8, **default 1**, and the live `.env` sets it to `1` — so no parallelism is active today. Sixteen agents hitting a 14B

> model concurrently on one GPU caused OOM and 60-second timeouts; the queue makes the cadence roughly one think every 3–4 seconds office-wide.

Loop 2 — the CofounderAgent director loop (top-down)

Every 15–30 s the CTO (Abby) builds a state-reactive prompt from live office state — who is idle, working, just shipped, or stuck — plus the shared event feed. One LLM call returns a JSON array of 2–4 commands (`speakTo`, `walkTo`, `callMeeting`, `delegate`). Abby is the only agent allowed to call group meetings or route around a stuck agent. After five consecutive errors she falls back to a scripted demo loop, so the office never goes silent. As of 2026-07-19 the cofounder path routes through the **main** tier rather than heavy.

Layer model

Layer	What	Storage	Git policy
Canon	`SOUL.md`, company brief, catalogs, sprite maps, city specs, source, migrations	Files in git	Commit
Runtime (durable, unversioned)	59 untracked JSON/JSONL files — the Grok/mentor learning loop, proposals, telemetry, job/email state	`denizen/data/`	**Do not commit**
Runtime (office)	Work output, telemetry, memories, CRM, threats, visitor assistants	Supabase/Postgres	Do not commit
Ephemeral	World-state positions, think queues, WS fan-out, temp worktrees	In-memory	Never commit

Note this is a **correction**: earlier versions described three layers and omitted the untracked runtime-file layer, which is load-bearing precisely because the database is optional. §8 gives the full six-store picture.

04 · The NPC system — SOUL, MEMORY, the think loop

`SOUL.md` — the identity contract. The full Markdown becomes the agent's system prompt. Two sections are parsed: `## Provider` selects the backend (`claude`, `lmstudio`, `lmstudio-fast`, `grok`, `kimi`); `## Role` is the short job title. Everything else — Core Identity, Personality, Values, Boundaries, Backstory, Opinions, Quirks, filler phrases, Idle Phrases — is concatenated into the prompt.

> **Why personalities are authored, not generated.** Abby's filler line is "Let me be direct about something," said right before the thing everyone was already thinking. Roki says "Got it. Okay." back-to-back when absorbing something new. These are written into the soul file, which is what keeps an agent recognizably itself across hundreds of conversations.

`MEMORY.md` — persistent episodic memory. Markdown plus tag conventions, appended as

`- [ts] entry`. Soft cap `DENIZEN_MEMORY_SOFT_CAP` 2,048 bytes triggers archive-compression (with a 30% aggressive prune on bloat); the hard trim at 200 lines keeps the last 150. Every write mirrors to Supabase.

Retrieval is keyword-based, not vector-based. `src/denizen-memory-retrieve.js` does **top-k = 8 TF/keyword-overlap** selection with a 1,200-char budget, and only engages when raw memory exceeds 80 chars (otherwise the last-N chars are used). Injection is capped at 280 chars, 180 in economy mode. The query is built from task, thought, directive, goal, room, chat and skill context.

⚠ **There are no embeddings in the memory path.** The embedding code path in `denizen-memory-retrieve.js:5-8` is explicitly optional, gated on `opts.useEmbeddings`, and **no caller passes it.** The embedding machinery that does exist is a *compute-mesh job type* (`src/denizen-embedding-worker.js:43`) that returns `{text, dims, sample}` and **never writes a vector store**. There is no pgvector, FAISS or HNSW anywhere in the repository. Any claim of semantic or vector memory retrieval is unsupported.

Tag	Format	Effect
SKILL	`[SKILL:name:+N]`	Increments a counter; counts ≥ 3 surface as "expertise" in the think prompt
FACT	`[FACT:Subject:claim]`	Added to a "Known facts" block; supports cross-agent awareness
TOPIC	`[TOPIC:tag]`	Indexes a memory for retrieval by conversation partner
OUTCOME	`[OUTCOME:ok|stuck|blocked|success]`	Used by the CTO to prioritize anyone stuck or blocked
MOOD	`[MOOD:event:+/-N]`	Biases future cycles (usually written by code)

The decision JSON contract.

```
{
  "reasoning":  "state, what matters most, why this action",
  "plan":       "one-line projected next 2 steps",
  "thought":    "internal monologue (one sentence)",
  "action":     "talk|work|collaborate|break|check|meeting|report|run_skill|…",
  "target":     "NPC name or null",
  "location":   "desk|breakroom|conference|storage|manager_office|reception|…",
  "message":    "what they say out loud (< 120 chars)",
```

```
  "taskPhase":  "starting|continuing|finished|none",
  "save":       "brief note written to memory",
  "outcome":    "ok|stuck|blocked|success|null",
  "skill":      "github|browserclaw|coding-agent|supabase-bridge|…",
  "skillParams": { }
}
```

The schema is placed in the **user** message — the last thing the model sees — not the system prompt. This produces materially better JSON compliance than schema-in-system-prompt.

Sampling is centralized. `src/inference-sampling.js:17-28` defines profiles by task kind: chat 0.8/0.95/0.05/1.1, presence 0.9/.../1.15, reasoning 0.6/.../1.05, code 0.2/0.9/0.02/1.0, decision 0.4/.../1.05. This replaced per-call ad-hoc parameters.

The mood system. Every agent carries live mood injected into each think prompt; mood shifts tone, decision and action choice. `stress` rises with consecutive work cycles, urgent directives and blocked tasks, falls with breaks and finished tasks. `satisfaction` rises with finished tasks, positive player chat and completed directives, falls with long idle. `focus` rises with uninterrupted desk time, falls with context switches, meetings and interruptions. All 0–100.

The directive inbox. Any agent or the REST API can assign a direct order. It is injected as a prominent `DIRECT ORDER` block at the very top of the target's system prompt — above its soul, above company context — so it dominates the next decision. On `taskPhase: finished` the directive clears and the agent walks to the assigner to report back.

```
curl -X POST http://localhost:8080/api/assign-directive -H "Content-Type: application/json" -d '
```

⚠ That endpoint is currently unauthenticated — see §3.

The core observability feature is the colored dot above each agent, updated every 500 ms with a task label beneath it: `WORKING · TALKING · BREAK · MEETING · ERROR · CHECKING · REPORTING · IDLE`.

05 · The roster & provider mapping

The roster is 18 agents. The single source of truth is `NPC_ROSTER_ENTRIES` at `src/npc-roster.js:8-27`, confirmed independently by `npc-capabilities.js` `BY_NAME`. The long-quoted "16" is hardcoded in *prose* — `src/generic-soul-pack.js:55` ("sixteen teammates," injected into every

generic-pack soul prompt) and `src/demo-scene.js:320` ("Sixteen agents, one office.") — and repeated across nine docs. Grok and Claude joined as in-world NPCs via migrations `20260624000002` and `20260625000001`.

⚠ **20 `SOUL.md` files exist on disk; two are orphans.** `npcs/marcus/` and `npcs/sarah/` are **not in the roster** — those display names are served by `conference_man` → "Marcus" and `conference_woman` → "Sarah" (`npc-roster.js:16-17`). The orphan files are never loaded, because `agent-souls.js:39-41` iterates `npcRoster.entries` only. Anyone counting with `ls npcs/` will get 20 and be wrong.

Agent	Folder	Role	Provider	Notable
Abby	`abby`	CTO	`lmstudio`	Sole conflict arbiter (`conflict-detector.js:82` hardcodes `'Abby'`); `min_tier: main`, `think→heavy`
Alex	`alex`	Developer	`lmstudio`	Leads engineering, GitHub skill
Bob	`bob`	Researcher	`lmstudio`	`min_tier: main`; research chain head
Jenny	`jenny`	Developer	`lmstudio-fast`	Pre-merge gate; dossier honesty-pass owner
Dan	`dan`	IT Support	`lmstudio-fast`	Incident responder for Bouncer escalations
Lucy	`lucy`	Receptionist / CEO email triage	`lmstudio-fast`	**Default owner for any unmapped skill** (`denizen-crm-npc-personas.js:170`); search/browser intents force-route here
Bouncer	`bouncer`	Security Guard	`lmstudio-fast`	`max_tier: fast`; threat-confirmation prompt rail; runs real `nmap`/`ss`/`tcpdump`/`tshark` in Kali
Marcus	`conference_man`	Project Manager	`lmstudio-fast`	Surfaces blockers — "Where does that stand."
Sarah	`conference_woman`	Product Manager	`lmstudio-fast`	"What problem does that solve" kills bad features early
Edward	`edward`	Backend Developer	`lmstudio`	`min_tier: main`; WS routing, think loop
Josh	`josh`	Frontend Developer	`lmstudio`	Phaser scene, UI, HUD
Molly	`molly`	QA Engineer	`lmstudio`	`VERDICT: PASSED\`	REVISE` gate
Oscar	`oscar`	DevOps Engineer	`lmstudio`	Inference queues, model swaps, health
Pier	`pier`	Data Engineer	`lmstudio`	Supabase pipeline, structured decision logging
Rob	`rob`	UI/UX Designer	`lmstudio`	Layout, hierarchy, contrast; dossier skinning
Roki	`roki`	Intern / Market Observer	`lmstudio-fast`	`max_tier: fast` but `kinds.chat/think = main`; watchlist owner
Grok	`grok`	xAI Researcher & Engineer	`lmstudio` / **grok-cli**	`reasoning→heavy`; anti-fabrication prompt rail

Claude	`claude`	Anthropic Verifier & Systems Mentor	`lmstudio` / **claude-cli**	`reasoning→heavy`; honesty rules in SOUL

Provider comes from the `## Provider` / `## Chat Provider` Markdown headers, parsed by `agent-souls.js:43-54` — **not a config file**. Default when absent is `lmstudio`.

⚠ Correction: the roster is local-first now, not Claude-first

Earlier versions mapped seven agents to cloud `claude-haiku-4-5` and two to cloud `grok-4`. **No agent uses a cloud provider as its soul-declared backend today.** Every one resolves to `lmstudio` or `lmstudio-fast`; Grok and Claude additionally carry **CLI bridges** (`grok-cli`, `claude-cli`) for chat and for the `grok_cli_chat` / `claude_cli_chat` mesh job types. Cloud keys still exist and the fallback chain still reaches Anthropic, but routine operation is local. This is a cost and privacy posture worth stating plainly rather than a regression.

Tier resolution — `src/denizen-harness/` (9 files, ~1,400 LOC)

The single gate is `complete()` at `src/denizen-harness/inference.js:53`.

Tier	Env pin	Live value (dev)	Live value (prod)
`fast`	`LM_STUDIO_FAST_MODEL`	`google/gemma-4-e2b`	same
`main`	`LM_STUDIO_MODEL`	`mistralai/ministral-3-14b-reasoning`	**`qwen/qwen3.6-27b`**
`heavy`	`LM_STUDIO_HEAVY_MODEL`	`qwen/qwen3.6-27b`	same
`reasoning`	`LM_STUDIO_REASONING_MODEL`	**unset** — falls through to the heavy chain, plus `enable_thinking`	unset

⚠ **Dev and production disagree on `main`** — Ministral 14B locally, Qwen 27B in production. `DENIZEN_HARNESS.md:61` documents only the production value.

Each tier walks a fallback chain (`tiers.js:65-104`): env pin → catalog match → probe result → pattern match → largest loaded → `models[0]`. An **anti-2B guard** promotes `main` if it resolves below 8B while a ≥14B model is loaded (`tiers.js:107-114`).

Per-NPC tier policy (`tierForNpc`, `tiers.js:158-176`): `min_tier` raises, `max_tier` lowers, and a `kinds.<kind>` pin wins absolutely. Eight NPCs are pinned in `config/denizen-harness.json:81-128`.

Kind → tier defaults (`tierForKind`, `tiers.js:127-136`): `presence/idle/ambient` → fast; `code/coding-agent` → heavy; `think/decision/reasoning` → main; `skill_args` → fast; else main.

Retry and timeout. Two attempts maximum, fixed 500 ms delay, **no backoff** (`denizen-inference-runtime.js:377,392`). On a retryable error it swaps to the tier fallback model and **caps `max_tokens` to 1024**. Completion timeout defaults to **120,000 ms** (floor 5,000); probe

timeout 4,000 ms. The harness layer itself does **not** retry — `complete()` catches once and rethrows with `e.harness` metadata.

Cost controls. Default `max_tokens` 512; think loop 360 (320 economy); chat 400; memory injection capped at 280 chars (180 economy); global think gap 8 s (12 s economy); skill cooldown 15 min; an economy mode that raises every interval; a hybrid-director mode that plans routine cycles algorithmically with **zero LLM calls**; and a 7-model cost-projection table in `src/cost-projector.js`.

⚠ Retired and outdated model IDs in code

Model ID	Status	Location
`claude-haiku-3-5-20241022`	**RETIRED — requests 404.** Also a malformed ID (the real one was `claude-3-5-haiku-20241022`). It is the **hard default for `DENIZEN_PUBLIC_DEMO_MODEL`** and is propagated to the NAS by two provisioning scripts	`prompt-tuning.js:43`, `npc-brains.js:860,953`, `infra/grok-claude-mentor-loop.py:79`, `infra/patch-denizen-nas-env.py:33`, `infra/provision-nas-server.py:141`
`claude-opus-4-5`	Legacy, superseded	`src/computer-use-agent.js:81`
`claude-sonnet-4-6`	Legacy, superseded by Sonnet 5	`scripts/claude-denizen-autonomy.js:11,66`
`claude-opus-4-7` as top pricing tier	Two generations behind; Opus 4.8, Opus 5 and Fable 5 absent from the table	`src/cost-projector.js:44`
`gemini-2.5-flash`, `gemini-2.5-flash-8b`	Old generation	`npc-brains.js:870`, `llm-telemetry.js:435`
`gpt-4o-mini`	Old generation (pricing table only)	`llm-telemetry.js:441`
`grok-4.3`	**Current — no change needed**	`npc-brains.js:883`, `cost-projector.js:51`

The first row is a live defect, not cosmetic: the public demo path's default model no longer exists.

`GEMINI_API_KEY` is dead configuration. `PRODUCTION_DEPLOY.md:29` instructs operators to set it, but **no server JavaScript reads it** — there is no Gemini provider. It survives only in a sprite-labeling script.

Souls: files vs database

Both are implemented. **Files are live for the operator org; the database is live for tenants.**

Path	Status
`npcs/<folder>/SOUL.md` + `MEMORY.md`	**Live** — the operator org falls through to files even with the DB enabled (`agent-souls.js:75-101`)
`agent_souls` table	**Real DDL, live for non-operator orgs** (`20260624000001_agent_souls.sql:4`)
Template cloning (`MULTI_USER_OFFICE_PLAN.md`)	**Live** — one endpoint, `server.js:7707`, default pack `1921-govcon`
`generic-office` template pack	`[PARTIAL]` — souls are **generated in code**, not files; `data/templates/generic-office/` holds only `business-model.json` and `company-brief.json`. ⚠

	`ROLE_BLURBS` has 17 keys — **`claude` is missing**, so generic-pack orgs get a fallback blurb for that agent

06 · Capabilities — skills, coding agent, chains, arbitration

Skill bridge — exactly 29 skills. During a think cycle an agent can return `action: run_skill`. The request routes through the Jelly runtime — which replaced the OpenClaw gateway as the default path — and the result is injected into the agent's next think cycle as a `## Skill Result` block.

`registry.list()` returns **29 = 23 native + 6 vendor**, logged at boot (`server.js:3128`).

Group	Count	Skills
Native (`jelly/skills/*/index.js`)	**23**	`browserclaw`, `build-skill`, `cloud-vm`, `coding-agent`, `consult-grok`, `consult-mentors`, `content-moderation`, `cursor-bridge`, `denizen-data`, `docker-sandbox`, `github`, `grok-telemetry-optimizer`, `health-check`, `job-pipeline`, `market-watch`, `pdf-dossier`, `platform-publish`, `remediate`, `research`, `security-linux`, `supabase`, `trade-suggest`, `trend-scraper`
Vendor (legacy subprocess)	**6**	`web-search`, `resume-generator`, `pdf-generator`, `canvas-render`, `lm-studio`, `screen-capture`
OpenClaw fallback	0	Gated on `JELLY_OPENCLAW_FALLBACK`, unset
NPC-authored approved	0	`denizen/skills/` is empty; no skill proposals on disk

Two counting traps worth recording so the number stays right: **do not say 24 native** — `jelly/skills/` has 24 directories but `cms-publish/` is empty and silently skipped. And **do not say ~100** — the 72 `denizen-data` action names are *aliases of one skill* and create no registry entries.

Gates on every invocation: contract validation and cooldown, then approval, then a `Promise.race` timeout. ⚠ Contracts exist for only **10 of 29** skills. Three names in `src/npc-capabilities.js` (`desktop-agent`, `screen-observe`, `visionclaw`) are **not Jelly skills** — they are intercepted by the station bridge before Jelly.

Coding-task-runner. `[DONE]` When an agent requests `skill: coding-agent`, the server runs Claude Code in an isolated git worktree, captures the diff, and writes a code-review proposal requiring human approval before merge. Guardrails: forbidden files (`npcs/`, `.claude/`, user-profile, company-brief — agents cannot edit their own soul/memory or system settings); max 5 files per task; max 1 pending proposal per agent. Flow: `git worktree add` → `npx @anthropic-ai/claude-code --print` → capture `git diff` → write proposal to `reports/code-reviews/` → broadcast `npc_code_proposal`.

Approval: `GET /api/code-proposals`; `POST /api/code-proposals/:id/approve` (merges, cleans worktree) or `/reject`.

Task chains. `[PARTIAL]` A chain is a sequence of `{npc, task, id?, gate?, skill?, artifactCategory?, reviseGoto?}` steps, run **strictly sequentially** — there is no parallelism. Each

step's artifact passes as context to the next agent, and the chain advances when the NPC's own `think()` emits `taskPhase: 'finished'`. A full pipeline reads Sarah (spec) → Rob (mockup) → Josh (implement) → Jenny (review) → Molly (regression test). Broadcasts `npc_chain_started` / `step_done` / `complete`.

Entry points: `POST /api/task-chains`, `POST /api/content-pipeline` (a 9-step preset), `POST /api/lucy-research-chain`, `POST /api/lucy-job-research-chain`.

⚠ Four honest limits: **there is no persistence** — chains live in in-memory `Maps` and are **all lost on restart **(`task-chain.js:10-11`);** the content gates fail open** — a missing verdict defaults to PASS (`content-gate.js:63,73,88`); retries exist only as revision loops capped at 3, with **no error/timeout retry, no backoff and no dead-letter path**; and `_npcActiveChain` is keyed by NPC, so starting a second chain for the same agent **silently overwrites the first**. Also note `step.skill` is only a *text hint* — steps become LLM directives, not skill calls.

`data/dossier-chain-template.json` is **dead data**: referenced only in prose, loaded by no code, and it declares no `id` or `gate` fields, so `reviseGoto` lookups would return `-1`.

Conflict detection & arbitration. `[PARTIAL]` Every artifact is logged in a rolling 50-entry buffer. When two agents produce conflicting outcomes about the same topic (same `chainId`, or ≥3 overlapping words longer than 3 characters, differing `outcome` fields, both under five minutes old), a conflict record queues for Abby — hardcoded by name at `conflict-detector.js:82`. Her next think prompt receives both positions under an `ARBITRATION NEEDED` block.

⚠ **There is no general arbiter.** Five unrelated mechanisms cover what the word implies: this conflict detector (Abby-only); static org-chart escalation via `reportsTo`; a static skill→owner table that **defaults everything unmapped to Lucy**; explicit assignment with **no contention resolution** (`delegate_task`, `assign_task` and `broadcast_task` all just call `assignDirective`); and one real compare-and-set on station tasks (the Cursor work queue is a JSONL read-modify-write **race with no file lock**). And `CofounderAgent` — despite the name — imports neither the agent bus, the task-chain runner, nor the skill runtime; it is a narrative director, not a dispatcher.

Proposal queues. `[DONE]`

Kind	Trigger	Proposes	Budget	Status
SOUL edit	Daily reflection	Add or drop one line in the agent's own `SOUL.md`	1 / agent / day	⚠ **generator is dead code**
New animation	`[ACTION:requestAnimation:...]`	A new sprite animation	2 / agent / day	Live path, **file empty**
New capability	`[ACTION:requestCapability:...]`	A new action verb	1 / agent / day	Live path, **file empty**

Dossier	`pdf-dossier` skill	A branded document, approval-gated before send	1 pending / agent	Live

⚠ **The SOUL reflection loop does not run.** `reflectOnDay()` (`npc-brains.js:6736`) has **zero callers repo-wide** — no scheduler, no cron, no route invokes it. The prompt builder, validator, applier and all three endpoints (`POST /api/soul-proposal`, `GET /api/soul-proposals`, `POST /api/soul-proposal/approve`) are real and live, and `data/soul-proposals.json` is `{"proposals": []}`. So the review-and-apply plumbing works but **nothing generates proposals.** `SOUL_REFLECTION.md` describes a daily loop that is not wired. `data/capability-proposals.json` and `data/animation-proposals.json` are likewise empty — those paths are live but unexercised.

> **Why the approval gate is non-negotiable.** Without it, agents gradually delete their personality quirks ("I'm anxious") in favor of bland competence. The gate exists precisely because the goal is not sixteen indistinguishable productivity bots — it is a recognizable team whose identities evolve under human review.

⚠ The honesty rail — the most important correction in this document

Earlier versions described `src/hallucination-guard.js` as an enforcing gate that agents "cannot bluff their way past." **That is not what the code does.** There is no single guard; there are 11 mechanisms across three tiers, **only two of which hard-block, and neither of those is about NPC speech.**

Mechanism	Tier	Behavior	Default
`hallucination-guard.js:55` `check()`	**advisory**	Returns `{flagged, note}`. No rewrite, no rejection, no throw	**never invoked** (see next row)
`npc-brains.js:5006-5037` grounding call	**logs only**	Queues a prompt note, broadcasts `npc_hallucination_flag`, logs `grounding: N unbacked claim(s)`. **The decision executes regardless** — fire-and-forget inside a sync method	**OFF**
`hallucination-guard.js:80-87` model verifier	**dead code**	Requires a `verifyFn`; **no caller passes one**. `buildVerifyPrompt` has zero consumers	never runs
Prompt-note injection	nudge, one cycle late	Injects the note, then deletes it (one-shot); the compact path truncates it to 200 chars	on if a note exists
`denizen-evidence.js:146` thought disclaimer	**advisory**	Appends a soft note — **the only live speech-side rail**	**ON**
`prompt-tuning.js:83` `evidenceRule`	prompt text only	"Do not claim you fixed/shipped/researched … unless you ran run_skill or produced an artifact this cycle"	⚠ **only on the non-compact path, and `compactThinkPrompt` defaults true**

Grok / Bouncer prompt rails	prompt text only	Anti-fabrication and threat-confirmation rules	on, hardcoded
`dossier-system.js:703` `runHonestyPass`	**HARD BLOCK + rewrite**	`ok:false` → no PDF or HTML written; rewrites unsourced figures to `[UNSOURCED:...]`	**ON**, bypassable via `force:true`
`denizen-node-client.js:142` `ranOk`	**HARD BLOCK**	Stops `{ok:false}` results being POSTed as mesh successes	**ON**

The kill switch. At `npc-brains.js:5006`:

```
const skipGrounding = decision._director
  || decision._parseFallback
  || (promptTuning?.realWorkMode === false);
```

And `prompt-tuning.js:86` has **both ternary branches set to `false`**:

```
realWorkMode: envBool('DENIZEN_REAL_WORK_MODE', economy ? false : false),
```

`denizen/.env:61` sets `DENIZEN_REAL_WORK_MODE=false`, and `.env.production` sets it neither way — so **production is also off.** Only `.env.production.template` and `infra/patch-denizen-nas-env.py` set it true. Even with real-work mode on, `decision._director` skips grounding on most cycles because the hybrid director defaults on. The application's own UI admits this — `llm-telemetry-panel.js:1195` reads `'unbacked claims — enable real-work mode to enforce'`.

Further gaps, all verified:

- **Player/CEO chat has no evidence hook at all.** The disclaimer helper is called only from `getResponse()`; `getPlayerResponse:5563` has none.

- **No claim-to-evidence matching.** The check is "did *anything* run in the last 3 minutes," so any skill run launders every completion claim in the window.

- **`npc_hallucination_flag` has no consumer.** It is produced and allowlisted but no client handles it — the flags are invisible to the operator.

- **The mesh guard treats "no result" as success** — `result === null` yields `ranOk === true`, and `{ok:true, mocked:true}` passes because the `mocked` flag is never checked.

- **The dossier gate is bypassable by the NPC's own parameters** (`force:true`).

- **No test file for `hallucination-guard.js`** — zero requires across `tests/` and `scripts/`. No test for the node-client `ranOk` guard either.

Detection itself is two regexes matching completion verbs ("i fixed/solved/completed/shipped...", "it

works", "done") and factual framings ("the data shows", "research shows", "according to", "\d+%").

Why this matters more than any other item here. The honesty rail is this project's brand signature — it is the reason the dossier system is safe to hand to an autonomous agent, and it is the stated mitigation for "agent hallucination" in the risk table. Today it is **an advisory logger that is disabled by default**, with the two real enforcement points sitting in the dossier renderer and the mesh client. Three fake-success bugs shipped in this period (652 mesh jobs reported complete, Discord music returning `{ok:true}` with no audio, and the consult-grok false-failure loop) — every one found by the operator, not by the guard. Turning `DENIZEN_REAL_WORK_MODE` on, giving the guard tests, and adding a consumer for the flag event are the smallest changes that would make the claim true.

07 · The intelligence spine — facts → outcomes → predictions

The intelligence spine is Denizen's durable learning loop: operational events become immutable facts, facts convert into outcomes (ground-truth labels), and outcomes continuously validate predictions.

```
Action happens     -->  record FACT (immutable)        denizen_facts

Time passes        -->  observe OUTCOME (label)        denizen_outcomes

Model runs         -->  store PREDICTION (validate)    denizen_predictions

                            |

   denizen_record_outcome() calls denizen_validate_predictions()

     --> backfills validated_at, actual_outcome_id, accuracy_score

     --> feeds prediction-summary + morning briefing cards
```

> **A crucial design rule.** Do not duplicate business truth. `intake_forms`, `rfps` and `clients` stay the source of record. Denizen links, enriches and logs facts about them — it never becomes a competing copy.

What is verified real. `[DONE]` All six core tables (`denizen_facts`, `_outcomes`, `_predictions`, `_entity_registry`, `_entity_links`, `_snapshots`); the functions `denizen_record_fact`, `denizen_record_outcome`, `denizen_validate_predictions`; all six documented triggers (intake, leads, rfps, clients, appointments, trade suggestions, plus job applications); runtime telemetry facts (`npc.work.*`, `llm.completed`, `llm.failed`); `runPredictionMaintenance` with Brier-style scoring; and `buildMorningBriefing`.

⚠ Three corrections that change what this section can claim

1. The loop is not closed in production. [PARTIAL]

infra/apply-migrations-nas.py:24-29 hard-skips four migrations because they depend on Dolonia platform CMS tables:

```
SKIP_VERSIONS = {20260606000005, 20260606000006, 20260606000012, 20260606000013}
```

The NAS runs 47 of 51 migrations. Missing in production: the outcome triggers, the funnel and velocity views, and the platform identity bridge. Status is observable at GET /api/platform-bridge/status; see NAS_PLATFORM_BRIDGE.md. Until those apply, facts accumulate but outcomes are not being labelled by trigger in the live deployment.

2. MiroFish does not exist. [PLANNED]

There is no MiroFish service, container, or infra script anywhere in the repository — only an HTTP client. MIROFISH_ENABLED is **off unless explicitly set**, so runEntityForecast short-circuits to _heuristicFallback before any network call. Even when enabled it health-checks, then falls back on any failure or a 10 s timeout. Default base URL is http://localhost:5001. **The integration is complete; the counterparty is imaginary, and heuristics are the only path in practice.**

The heuristic is honest about itself — modelKey: 'heuristic_v0', metadata.source: 'mirofish_fallback', fallback_reason: 'mirofish_unavailable' — but note it is **implemented twice**, once in SQL (…000006.sql:167-212) and once in JS, with identical weights, no shared source, and both stamped heuristic_v0 / 0.2.0. They will drift.

3. The ~82.5% prediction-accuracy figure is withdrawn. Given (1) and (2), any accuracy number currently derivable is scored against heuristic predictions in a deployment where outcome triggers are not installed. The figure may well be reproducible — but it needs re-derivation with stated provenance before it appears in an external document. Until then this document does not quote it.

Fact writers by surface

Surface	Mechanism	Example fact types
Website intake	`intake_forms` trigger	`intake.submitted`, `intake.status_changed`
Leads	`leads` trigger	`lead.submitted`, `lead.status_changed`
RFP pipeline	`rfps` trigger	`rfp.ingested`, `rfp.bid_decision_set`
Client lifecycle	`clients` trigger	`client.created`
Appointments	`client_appointments` trigger	`appointment.booked`, `appointment.status_changed`
Job applications	`denizen_job_applications` trigger + service	`job_application.created`, `job.interview`, `job.offer`

Trade suggestions	`denizen_trade_suggestions` trigger	`trade_suggestion.created`
NPC / runtime telemetry	`src/denizen-db.js`	`npc.work.*`, `llm.completed`, `office.*`, `visitor.*`
MCP memory writes	`update_memory` tool	`npc.memory.updated`

Prediction & pipeline API surfaces

Endpoint	Method	Purpose	Status
`/api/intelligence/prediction-summary`	GET	Prediction counts, validation status, accuracy, typed breakdown	`[BROKEN]` — always returns `topForecasts: []`, see below
`/api/federal/pipeline-summary`	GET	RFP counts by status, pipeline value, upcoming due dates	`[BROKEN]` — no org filter, see §10
`/api/intelligence/mirofish-status`	GET	External forecast availability & config	`[DONE]`
`/api/intelligence/mirofish-sync`	POST	Pull/sync external forecasts into `denizen_predictions`	`[DONE]` (no counterparty)

Why `prediction-summary` returns no forecasts. The live handler is `server.js:7730`, and it calls `getPredictionSummary(orgId, db)` **with no options object**, so the `includeTopForecasts` branch at `denizen-prediction-loop.js:147` never runs and `topForecasts` is always `[]`. The morning briefing is unaffected because it passes the flag correctly.

⚠ Worth recording precisely, because it is easy to misdiagnose: **`src/route-registry.js` and `src/routes/intelligence-routes.js` are both entirely dead code.** `route-registry` is required at `server.js:171` and **never called** — there is no `registerDenizenRoutes` invocation, not even a commented one. All four intelligence endpoints live inline at `server.js:7723-7777`. So `FEDERAL_PIPELINE.md:66` ("registered via `src/route-registry.js`") and `MIROFISH_PREDICTION.md:20` ("exposed by `src/routes/intelligence-routes.js`") are both false, and `SERVER.md:424` is wrong that the registry call is "commented out" — it is simply an unused import. Fixing the bug means changing `server.js:7730`, not the registry.

Also: `getPredictionSummary` reads at most 1,000 rows and recomputes per-type accuracy with a nested `filter` inside a `for` loop — O(types × rows). Prediction maintenance runs on a **6-hour interval** (`server.js:13134`, fired once immediately at boot). **MiroFish sync has no scheduler at all** — it is reachable only from `POST /api/intelligence/mirofish-sync`.

One naming collision to avoid: `src/intelligence-loop-health.js` is **not** part of this spine despite the name. It reads Grok mentor/consult JSONL files from `denizen/data/` and touches no `denizen_*` table.

08 · Database architecture — 6 stores, 77 tables, 51 migrations

Migration strategy — "forever updating." One migration per change; never edit old SQL. **51

ordered migrations** exist (2026-06-06 → 2026-07-19), convention `YYYYMMDDHHMMSS_snake_case.sql`, each idempotent (`CREATE TABLE IF NOT EXISTS`, `ON CONFLICT DO NOTHING`), self-tracked via `denizen_schema_migrations`. Additive-only in production: add columns with defaults, add tables, never drop in place. Event-sourced logs are append-only (`denizen_facts`, `llm_events`, `npc_memories`). Every server boot writes a `denizen_office_sessions` row, and work rows carry `office_session_id` so restarts do not duplicate analytics.

77 distinct tables are created here, plus **13 externally-owned platform tables** that migrations only `ALTER` and never create (`intake_forms`, `leads`, `clients`, `projects`, `tickets`, `rfps`, `workflow_runs`, `documents`, `key_value`, `logs`, `client_appointments`, `subscriptions`, `room_bookings`). Other objects: 4 views, 0 materialized views, 24 distinct functions, 7 triggers, 102 indexes, 80 `ALTER TABLE`, 0 enums.

Family	Tables
Foundation	3 — `denizen_schema_migrations`, `_office_sessions`, `_npcs`
Work & telemetry	5 — `_work_output`, `_artifacts`, `_office_events`, `_npc_memories`, `_llm_events`
Office layout / game	7 — `_office_layout`, `_office_progress{,_events}`, `_npc_stats_daily`, `_office_quests`, `_office_achievements`, `_station_tasks`
Intelligence spine	8 — `_organizations`, `_org_members`, `_entity_registry`, `_entity_links`, `_facts`, `_outcomes`, `_predictions`, `_snapshots`
CEO / pipelines / governance	11
Visitor	12
Identity / auth	4
NPC teams	4
Souls / business model	2 — `agent_souls`, `org_business_models` (unprefixed)
CRM	2 — `_crm_workspaces`, `_crm_portal_tokens`
Compute mesh / credits	5
MCP	2 — `_mcp_tokens`, `_mcp_audit`
Trading	2
Security	1 — `_bouncer_runs`
Discord	3 — `_guild_plans`, `_guild_pet_entitlements`, `_discord_guild_orgs`
Dolonia conversation	6 — `dolonia_conv_*`, `dolonia_products`

Multi-tenant model & access — with a correction

The spine is org-first. Tenant-isolated tables carry `org_id`, and reads are enforced through RLS helpers from migration `000012`: `denizen_auth_user_org_ids()` resolves which orgs a signed-in user can access; `denizen_user_can_access_org(org_id)` is the boolean predicate used in policies. Protected

agents (`abby`, `bouncer`, `lucy`) are flagged `protected = true` and cannot be claimed or issued MCP tokens; the demo org (`acme`) is blocked from business data.

⚠ **"Org-scoped RLS everywhere" is not accurate as stated.** RLS is enabled on **37 of 77 tables — 40 have none**, and 45 `create policy` statements are preceded by 49 `drop policy` statements, so the net live policy set depends on which migrations actually ran. Several of the 40 are legitimately non-tenant (`denizen_schema_migrations`, catalogs), but the blanket claim should be replaced with the count and a list of deliberate exemptions. Additionally, the service-role key **bypasses RLS by design** (`denizen-db.js:10`), which is how the cross-org leak in §10 became possible.

Six storage layers

#	Store	Contents
1	Git-committed canon	125 tracked files in `denizen/data/`, 20 × `npcs/*/SOUL.md`, city specs
2	**Runtime-mutated local JSON/JSONL**	**59 untracked files** — the whole Grok/mentor learning loop, proposals, telemetry, job/email/research state. Load-bearing because the DB is optional
3	Postgres `public` — `denizen_*` / `dolonia_*`	77 migration-owned tables
4	Postgres `public` — platform	13 externally-owned tables, altered only, owner `supabase_admin`
5	**Postgres `city` schema (NAS only)**	`city.limezu_assets`, `city.build_state` — created by `infra/setup-city-db-nas.py`. **Referenced nowhere in `denizen/`** — write-only build memory, not wired into the app
6	In-memory	`WorldStateRegistry` / per-org `WorldState`, think queues, WS fan-out, skill cache

Database access is gated by `SUPABASE_URL` + `DENIZEN_DB_ENABLED` with the service-role key preferred (`src/denizen-db.js:10-57`). With the DB disabled the app degrades to layers 1, 2 and 6.

Three defects in the migration chain

1. **Version collision.** `20260707000001_denizen_bouncer_runs.sql` and `20260707000001_npc_stats_and_office_progress.sql` share a version prefix and both insert that version row. The runner's `if ver in applied: skip` (`apply-migrations-nas.py:129`) then **silently skips whichever runs second.**

2. **Two migrations never record a version row**, so they re-run on every apply: `20260714000001_discord_identity_link.sql`, `20260716000001_dolonia_conversation_data.sql`.

3. **Duplicate index with conflicting definition.** `20260714000001` creates `denizen_user_identities_discord_user_id_idx` as **UNIQUE**; `20260715000001` creates the same name **non-unique**. Both use `if not exists`, so apply order decides which constraint you get.

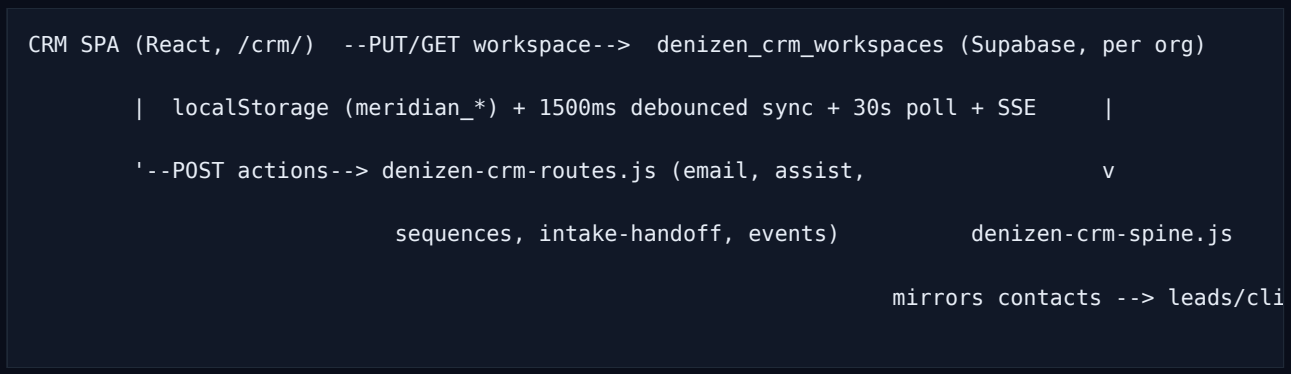
DATABASE_SCHEMA.md is substantially stale: **31 created tables are undocumented** and **7 documented tables are never created** (two of those exist as *files* instead —

`data/npc-directives.json`, and mood is in-memory). Its build-order section stops at migration `000006`; 45 landed after.

09 · Meridian CRM — the embedded sales operating system

Meridian (codename "Deal Weaver") is a full sales CRM embedded inside Denizen — a React SPA (`denizen/crm/vendor/`, built to `crm/dist/`) opened from the in-office phone or at `/crm/`.

The truth model is a single JSON blob per org, not relational tables. `denizen_crm_workspaces` has PK `org_id`, a `state` jsonb column, `schema_version`, `updated_at`, `updated_by`; RLS is deny-all and the server uses the service-role key. `state` is whitelisted to exactly **34 keys** (`src/denizen-crm-store.js:12-47`) — anything else is silently dropped by `sanitizeState`. Four of the 34 are undocumented: `supportTickets`, `marketingCampaigns`, `teamPolicy`, `usageLog`.



Sync. 1,500 ms debounced `PUT /api/crm/workspace`, 30 s poll, SSE push via `GET /api/crm/workspace/stream`. Optimistic concurrency: the client sends `ifUpdatedAt`; on conflict the store returns `{conflict:true}` plus server state and **the client discards local edits and applies server state**, flashing "conflict" for 2.5 s. On every save the store runs automations → audit-diff → upsert → fire-and-forget spine sync, portal-token sync and webhooks.

Who writes. Humans via the SPA; NPCs via `denizen-crm-agent-writes.js` → `denizen-crm-mutations.js` behind fail-closed org policy (**7 of 16 personas** have `canWrite:true` — abby, lucy, conference_man, conference_woman, molly, pier, rob); NPC natural language via `denizen-crm-capture.js`; MCP clients via 4 read/draft-only tools; and the server itself (automations, scheduler, digest, spine backfill, SAM mirror, Gmail inbound). Auth `verifyCrmSession` accepts a visitor session, a demo session, **or** an `x-denizen-ceo-grant` header.

Route count disagrees three ways. 35 real handlers exist; `GET /api/crm/health` self-reports **22**; the architecture doc lists **20**. The 13 the health diagnostic omits are quote ×4, resend-webhook, stripe-webhook, stripe/status, and calendar ×6. Because `/api/crm/health` exists specifically as a **deploy diagnostic**, that gap is a live defect, not doc drift.

What is real, and what is not

Real `[DONE]`: contacts/deals/tasks/notes/companies CRUD with cloud sync and spine mirror; pipeline kanban with stage changes, deal comments and email engagement; packages, quotes and client PDF generation; email templates and compose via Resend with mailto fallback; sequences (server scheduler ticks each step, 5-min tick); intake workflow plus public `POST /api/crm/intake`; AI Assistant with persisted threads and a `source:'rules'` fallback when no provider is configured; analytics with daily metric snapshots (90-day ring, written at 01:00 server-local); **13 automation triggers** (docs undercount at 9) with the documented 5 actions; outbound webhooks; client portal `#/portal/:token` with e-sign and Stripe Checkout; Google/Microsoft calendar OAuth; CSV import/export, JSON backup/restore, duplicate merge; data-health panel, audit-log viewer, per-contact GDPR export; team invites; **41 industry templates; weekly digest Monday 09:00 server-local;** Gmail → CRM inbound** (3 routes, real, undocumented); and the **Support** and **Team** tabs (real, undocumented).

Not real, and currently presented as real:

Surface	Status	Evidence
AI Prospector	`[UI-ONLY]`	`AIProspector.tsx:16` `MOCK_RESULTS`, `:72-75` `setTimeout(..., 4000)` fake scan. The UI text admits "we simulate a web scan" but `USER_GUIDE.md:399-407` presents it as functional and neither doc marks it UI-only
Marketing campaign launch	`[PARTIAL]`	`MarketingTab.tsx:73-80` only flips status to `active`. **It sends nothing.** Persistence and automations are real
"AI Probability" (Pipeline)	`[DONE]`, mislabeled	`DealProbabilityAI.tsx:21-63` is a client-side additive rule score — no model, no LLM call

The UI has **21 nav items**; the user guide documents 18, and its keyboard 1–0 shortcut map is wrong from key 6 onward because Support, Marketing and Team were inserted into `NAV_ITEMS`.

> **Why this matters for the whole product.** Meridian is the bridge between "interesting agent demo" and "software a business pays for." The agents get a real system of record to act on; the CRM gets an autonomous team that works the pipeline. Each makes the other monetizable — which is exactly why the UI-only surfaces need honest labels before this document goes to a buyer.

09b · Customer product objects — Floor, Workbench, Buzz (+ Seat an AI, Studio Board)

The public story on `landing.html` is three objects operators can point at, not sixteen internal subsystems. v1.8.0 records them because the July dossier never named them and the office chrome now ships them.

Object	What the operator sees	Status	Primary code
The Floor	Pixel office — agents at desks, status dots, walk-through observability	`[DONE]`	Phaser office + roster
The Workbench	Orchestrator rail for Abby / Seat an AI / handoffs	`[DONE]`	`src/work-rail.js`, `/api/workbench`
Buzz	Capture ring → note card → `/api/context-drop` to Abby (screenshot, record, dictate, meet)	`[DONE]`	`src/buzz-capture.js`, `src/context-drop.js`, desktop overlay
Seat an AI	Drop Claude / ChatGPT / Cursor / Grok / LM Studio onto a desk	`[DONE]`	`src/seat-ai.js`, `/api/seat-ai`
Meridian CRM	Embedded sales OS (§09) with member-scoped records + team activity viewer	`[DONE]` / `[PARTIAL]` deploy	`src/denizen-crm-*`, PR #67
Studio Board	Monday-parity agency board on the phone + MCP tools bound to CRM	`[DONE]`	`src/denizen-studio-board-*`, MCP studio/agency tools

Buzz honesty notes. Desktop Buzz can stay out of screen shares ("leave invisible"). On phones, display-capture APIs are skipped: Screenshot opens the photo/library picker on the same tap gesture; Meet falls back to mic-only recording; the ring no longer auto-closes on touch `pointerleave` (PR #73). Do not claim mobile screen-share capture.

Landing ↔ office. PR #72 shipped the landing promises into the office so demo links (`?buzz=screenshot`, `?seat=1`) hit real surfaces instead of marketing-only HTML.

10 · Integrations — Jelly, MCP, email, federal, market

Jelly — the skill runtime `[DONE]`

Jelly is Denizen's in-process skill runtime and dispatcher; it became the default execution path for agent skills, replacing the external OpenClaw gateway (lower latency, fully controllable, no separate service). `jelly/index.js` exposes `createJelly()`; `skill-executor.js` runs skills natively in-process or via legacy subprocess; a secure Supabase bridge and a BrowserClaw service run as sidecars. Business data actions go through `jelly/lib/denizen-data-actions.js` behind `assertBusinessActionAllowed`. The Jelly mascot — the bioluminescent jellyfish — is the project's brand mark.

The MCP surface — 38 tools across two gateways `[DONE]`

This is the largest single expansion since v1.6.0, which documented **three** tools. Both gateways are **Streamable-HTTP JSON-RPC only — there is no stdio server and no SSE transport.**

	Per-NPC	Team
Mount	`/mcp/npc/:npcKey` (`server.js:3850`)	`/mcp/team` (`:3859`)
Protocol	`2024-11-05`, serverInfo `denizen-npc-<key>` **v2.0.0**	`2024-11-05`, `denizen-team` v1.0.0
Auth	Bearer header or `?token=`	same

Rate limit	`DENIZEN_MCP_RATE_LIMIT`, default 120/token/hour	**none**
Purpose	one human ↔ one claimed NPC	org operator; `run_as_npc` into any non-protected NPC
Tools	**22 baseline**, 25 `oscar`, 27 `roki`, 29 `grok`	**20** + **8 MCP resources**

Composition: 9 core + 5 team + 4 CRM + 4 Cursor-bridge, then +5 market (Roki), +3 mesh (Grok/Oscar), +4 mentor (Grok). Team-only: `team_status`, `run_as_npc`, `broadcast_task`, `clear_directive`. Resources: `team://{roster,handoffs,standup,cursor-work}`, `telemetry://fleet/latest`, `telemetry://fleet/autonomy/backlog`, `mentor://dialogue/pending`, `telemetry://grok/consultations/pending`.

Six scopes: `read` (24 tools), `skills`, `crm_write`, `handoff`, `delegate`, `compute`. Presets: `read`, `standard` (read+skills+handoff), `full`. Tokens are `randomBytes(32).base64url`, stored as SHA-256 hashes only, **plaintext returned once**. Resolution via `denizen_resolve_mcp_token` (SECURITY DEFINER) rejects tokens under 16 chars, revoked tokens, the `demo` org tier, and protected NPCs. `tools/list` is **pre-filtered by scope**, so out-of-scope tools are invisible rather than merely refused. Every call — allowed or denied — audits to `denizen_log_mcp_audit`, wrapped so audit failure never blocks a tool.

⚠ **Four auth caveats that belong in the honesty section, not the feature list:**

1. **1-click OAuth always mints `full` scope** — `denizen-mcp-oauth.js:294` passes no `scopes`.

2. **The "Read only" preset can write.** `update_memory` and `log_activity` are mapped to `read`, so a token the UI labels read-only can still write NPC memory and work-feed rows. Intentional, but the label misleads.

3. **Empty scopes mean full access in JS but only 5 scopes in SQL** — the SQL fallback omits `compute`, so legacy tokens minted before that scope existed are permanently locked out of `enqueue_compute_job`, contradicting the documented "missing scopes = full access."

4. **`clear_directive` is absent from `TOOL_SCOPE`**, falling through to the `|| 'skills'` default.

Memory sync `[PARTIAL]` is dual-write out, batch-merge in. Three producers write to `denizen_npc_memories`: the in-world brain (which also writes `MEMORY.md`), MCP `update_memory` (which **does not**), and `mentor_finalize`. Inbound, `refreshDbMemories()` pulls the newest 40 rows per NPC and prepends them as `## Synced memories (database)`. **It runs on only two triggers — server boot and `mentor_finalize` — with no polling.** So an `update_memory` from ChatGPT or Cursor is durable in Postgres and visible via `get_status` immediately, but **the in-world NPC's prompt does not see it until the next server restart**, and `MEMORY.md` never receives MCP-sourced memories at all.

The ChatGPT path. ChatGPT Plus cannot take an MCP URL, so a REST/OpenAPI shim covers the same

scope-checked `callTool`: `GET /api/npc-sync/openapi.json` (OpenAPI 3.1, v2.0.0), 10 routes, same bearer and audit. `data/templates/chatgpt-npc-assistant.yaml` is the copy-paste Custom GPT config. Commit `d25bc7a2` made this path fast: the GPT used to call `assignNpcTask`, which queued the slow local brain and returned nothing useful. That is now **blocked** for ChatGPT clients, replaced by an in-memory job store (`skj_…` ids, 500-job cap, 30-min TTL), a `wait=false` async mode on `run_skill`, and a `get_skill_job` polling tool. REST defaults to `wait=false, max_wait_ms=20000` because Custom GPT Actions time out near 45 s.

Connect flow. `visitor-mcp-connect-ui.js` offers three entry points and a 5-tab modal defaulting to **1-Click** (MCP URL only, browser OAuth): 1-Click / Companion / Cursor-API / ChatGPT-GPT / Claude.ai. Every tab has Test connection, Rotate token, Done. Rotation is revoke-then-mint in the UI; there is no server-side rotate endpoint.

Lucy — email & job pipeline `[DONE]`

Email. Inbound arrives via an n8n Gmail watcher → `POST /api/email-inbound`. Lucy composes a reply. With `EMAIL_AUTOSEND=true` and no escalation keyword she sends automatically; otherwise she drafts. Escalation keywords always force draft-only: contract, nda, payment, legal, invoice, lawsuit, settlement, confidential, attorney, "urgent: ceo only." Set `EMAIL_INBOUND_SECRET` so n8n must include `?secret=…` or the request is rejected 401.

Job pipeline (7 steps): scrape → filter (role/salary/remote) → tailor resume → cover letter → apply (dry-run or live) → log → summary.

Federal contracting & SAM.gov `[PARTIAL]`

New since v1.6.0: a live federal opportunity feed. `GET /api/crm/sam/search` and `POST /api/crm/sam/ingest` (both session-required, because one call fans out to ~21 upstream requests on a shared key) hit `https://api.sam.gov/opportunities/v2/search`. Nightly auto-ingest is **on by default** — 24 h interval, 2-day lookback, limit 20, first run +3 min, kill switch `SAM_GOV_AUTO_INGEST=0`.

Two API quirks are load-bearing: **`typeOfSetAside` and `ncode` are single-value** — a comma list returns zero results — so the client fans out one request per combination, capped at `MAX_COMBOS = 24` with concurrency 4, merging and deduping by `noticeId`, and only throwing if *all* combinations fail. Dates must be `MM/dd/yyyy`. Ingest dedup keys on `noticeId`, else `solicitationNumber|title`, pre-read chunked at 80 ids; **a failed dedup read aborts the whole ingest deliberately, because there is no DB unique constraint on `sam_gov_id`.** `mapSetAside` tests `EDWOSB` before `WOSB` because the latter is a prefix of the former. Mirrored deals enter the GovCon stage `RFP Released`, one of nine stages from

Market Research to Protest.

⚠ **Three honest limits:**

- **Cross-tenant leak.** `federal-pipeline.js:192-198` queries `rfps` with `filters: {}`, and the service-role key bypasses RLS — so **any org passing the tier gate sees every org's RFP count and total pipeline value** in its morning briefing. Also affects `getRfpDetail`.

- **Nothing in the repo writes `rfp_requirements`.** Requirement review is real code but read-only over an externally-populated table, so SAM-sourced RFPs show `requirement_completion_pct: 0` forever.

- **Zero SAM test coverage** — no `*sam*.test.js` anywhere, despite the date, dedup and set-aside mapping logic being the most fragile part of the integration.

Commit `acb3a899` fixed a silent one-line failure here: `_loadLatestPredictions` filtered on `prediction_type: 'rfp.bid'`, but the trigger stores `'rfp.bid_probability'` — so **bid-probability enrichment returned `null` for every RFP** across `getRfpDetail`, `rfp-status`, `rfp-requirement-review` and the mirrored deal.

The org-gated briefing strip is `[DONE]` and gated twice: server-side `_sessionCanViewFederalPipeline` (false for `tier==='demo'` or `org_slug==='acme'`) and a client-side `isTeamSession()` re-check before rendering.

Robinhood — read-only by design `[PARTIAL]`

> **Hard safety rule in the code.** Denizen never executes trades. There is no order-placement API.

> Trade intent is stored as suggestion text only; the human executes any real trade manually.

> `placeTrade()` does not exist.

Roki owns market telemetry. OAuth uses PKCE over MCP Streamable HTTP with dynamic client registration; quotes fall back to a free public feed when Robinhood MCP access is unavailable. The flow is built but needs a one-time browser login to activate. ⚠ All 17 market routes are currently unauthenticated (§3).

11 · The Discord product — bot, Spaces, and the paid tier

New since v1.6.0 and entirely absent from it. By commit volume this was the largest work area in the period (67 commits), and it carries the platform's first real recurring billing.

Bot architecture `[DONE]`

`denizen/discord-bot/` is **70 files / 20,431 LOC**. Entry `index.js` (298 lines), `discord.js ^14.21.0`, **`9 gateway intents (3 privileged), 4 partials, per-guild JSON config in data/guilds/`**.

It is not a standalone service. The main Denizen server spawns it as a child process

(`server.js:13092-13131`), gated on `DISCORD_BOT_TOKEN` being present and `DENIZEN_DISCORD_BOT != off` (default `auto`), with 5 restart attempts and `10s × attempt` backoff that resets after 60 s stable. `uncaughtException` exits 1 so the supervisor restarts clean. A presence API listens on `127.0.0.1:5180` with **no authentication — the loopback bind is the only control**; org gating lives upstream in `assertMapAccess`.

There is no sharding. A single `Client`; only `ShardResume` is handled, to re-seed voice-channel occupancy.

Modules and commands

22 module ids across 20 directories. Highlights: moderation (22 commands, case-logged), music (15, Lavalink v4), **slots (14 — seller slots)**, community (12), misc (11, including `,eat` and map social), economy (10, Denizen Credits wallet), security (7, antinuke/antiraid/fake-permissions), denizen (7, account and map linking), avatar (5, layered character closet), plus tickets, voicemaster, pvp (button duels), minigames, shop, canvas, logging, hangout, automod, integrations, and `data-farm` (registered as `sense`, which harvests guild chat to tune *communication style*, not demographics).

130 registered commands — verified twice, by the manifest (`commands.json:4`) and a live registry boot, which agree. 182 distinct prefix names including 52 aliases; **111 slash-capable**; 19 prefix-only by design. Prefix is `,` with per-guild override. Permissions across the 130: 82 Everyone, 14 Manage Guild, 12 moderate-members, 6 server-owner, 3 Administrator, 3 ban-members. Note `,slots` alone fans out to 19 sub-actions, so the user-visible surface is much larger than 130.

⚠ **The slash budget is over and mis-trimmed.** `[BROKEN]` Discord caps an app at **100 commands per scope**; the bot has 111 slash-capable, so 11 are always dropped. The live API shows all** 7 guilds at exactly 100**, dropping the game commands — which is the intended `SLASH_TRIM_MODULES` choice.** But `DISCORD_SLASH_TRIM` is not set in `denizen/.env`.** Simulating current code with trim off drops a different 11 by alphabetical tiebreak: `unreactionmute`, `untimeout`, `urban`, `vanityrole`, `voicemaster`, `volume`, `warn`, `warnings`, `webhook`, `welcome`, `youtube`. So the next command sync silently swaps which 11 lose their slash form — trading deliberate game-command cuts for **staff and high-traffic commands**. Only 3 commands carry `priority: 5`, not enough to steer the tiebreak.

The paid tier `[DONE]`

Item	Value
Tiers	`free` / `pro` / `ultra`, per **guild** (PK `guild_id`), table `20260718000001_discord_guild_plans.sql`, RLS service-role-only
Prices	**Not in code** — only Stripe Price ID envs (`STRIPE_PRICE_PRO`, `_ULTRA`, `_PET`)

Premium modules	**Exactly two**: `security`, `community`. Avatar is **not** premium
Gate 1	Command dispatch checks `isModuleAllowed`; denial replies with `DISCORD_UPGRADE_URL`
Gate 2	`gatedHook()` wraps premium modules' **passive listeners** so antinuke sweeps and antiraid join checks don't run unpaid. Fails closed
Gate 3	Avatar cosmetics are separately **stripped** by `enforceAvatarPlan()`, re-run after role cosmetics "so free users can't keep paid drip"
Active	`plan !== 'free' && status ∈ {active, trialing}`, **plus a grace window** — `past_due`/`unpaid` still count while `current_period_end` is future
Cache	60 s TTL; a failed refresh **keeps last-known-good** so a DB blip won't downgrade a payer; no cache + no DB → free (fail-closed)
Endpoints	**3**: `POST /api/discord/billing/{checkout, pet-checkout, webhook}`
Webhook	Own HMAC verify, 300 s tolerance, raw body required
Downgrade	`subscription.deleted` → `plan:free, status:canceled`; missing `guild_id` metadata falls back to `findGuildBySubscription()`; `syncProGrantPool(false)` reclaims the 3 pet grants
Integrity	`stripe_subscription_id` has a UNIQUE partial index, so a replayed event cannot entitle a second guild

Pets are a separate per-member entitlement with a one-time Stripe price; active Pro/Ultra guilds get a pool of 3 concurrent grants. A **home-guild bypass** (new, uncommitted) gives one named guild `{plan:'pro', status:'active', source:'home_guild'}` with no Stripe and no DB read.

Seller slots `[DONE]`

"Seller slots" is SlotBot-parity monetization **for a host server's own resellers**, not for Denizen. A marketplace guild sells time-boxed private seller channels: each holder gets one channel where only they can post, with a finite `@here`/`@everyone` ping quota that resets on a configurable interval. It is a **free module** and its replies are deliberately public. 19 sub-actions split into staff and public sets; storage is per-guild JSON with atomic tmp+rename and a 500-entry history ring — **no database**. Auth is in-handler (`ManageGuild` or a configured staff role; `setup` additionally requires Administrator). Hourly expiry sweep and hourly ping-quota reset.

Spaces — the walkable map `[PARTIAL]`

The front end is `denizen/discord-map/` — a single **8,473-line `game.js`** plus a gate UI. Shipped: elevators in tall towers, door families with open textures and swing, colored chat bubbles, 14 animation rows, room recipes, food interactables, seat/bed surface heights, plaques, parallax sky, and **Phase G map social** (reactions map to poses, `,eat` yields held food, 5 s TTL dedupe). Thirteen districts, a full Halloween biome, room kits, sky traffic, and crowd-aware room sizing landed in July.

Honest gaps: ambient street pets were **deliberately reverted** in favor of paid per-member companions; the Phase E needs/hunger UI is **dead code** (`startBlockLife()` has an unconditional early `return`); the free-plan **watermark is computed server-side and ignored by the client**; the planned 8-module

`map/*.js` split never happened; and the catalog rebuild target is unmet (320 of 2,115 assets used versus a 600 goal).

⚠ **Licensing item requiring an operator decision.** The replacement pet art is labeled `NON-COMMERCIAL` in code (`discord-map/game.js:852`) while pets are a paid Stripe SKU.

Map access gating `[DONE]` funnels through one choke point, `assertMapAccess()` (`src/denizen-map-access.js:104`), called by both `/api/discord/map/access` and `/presence` so status and enforcement cannot drift. Order of checks: no `guildId` → allow; **`!guildOrg.dbReady()` → allow, so no Supabase means every map is fully open; Discord membership wins over everything** (viewer's Discord id resolved from the server-side session only, never a client-supplied id, 12 s cache, fails closed); signed `map_grant` (HMAC, 12 h TTL, guild-bound); Denizen team session in the same org; else 403 `TEAM_CODE_REQUIRED` or `MEMBERSHIP_REQUIRED`. Note this **inverts** the documented behavior — membership is now checked *before* the team-code gate, so a linked member gets in with no code and no grant.

External dependencies

Subsystem	Dependency	Degrade behavior
Music	Java **Lavalink v4** at `127.0.0.1:2333`	Infinite 5 s reconnect if down; graceful in-Discord error with setup help
Voice transport	one of `sodium-native` / `libsodium-wrappers` / `@noble/ciphers`	`[BROKEN]` **None installed.** `entersState(Ready)` throws, the catch falls back to a no-op `voice.setChannel()` and **still returns `{ok:true}`** — so `,play` says "Now playing" with no audio
Last.fm	`LASTFM_API_KEY`	Fully graceful
Upload notify	**YouTube Data API v3** (not Last.fm), 15-min poll	Silent no-op without a key
Avatar bake	`jimp` — lazy-required and **not declared** in `discord-bot/package.json`; resolves up to `denizen/node_modules`	Closet preview fails; defaults never 404
Entitlements	Supabase + Stripe	No Supabase → premium denied (fail-closed); no Stripe key → `stripe_not_configured`

12 · The city & world system

New since v1.6.0. A data-driven, multi-district LimeZu city, live at runtime via `?world=full`.

Format. `data/city-specs/` holds **40 files** — 19 JSON (17 district fragments `d1–d17`, a generated `d00-structure.json`, and `scene-specs.json`), 18 PNG previews, 3 Markdown. Fragment layers: `patches, roads, stamps, anims, animBands, solids, labels, portals, npcs, vehicles`. Coordinates are world pixels; native tile is 16 px; street width 64 px; stamp `origin ∈ bl|tl|bc` anchored at the floor baseline.

Merged output `data/city-plan.json` (241,851 B): **worldSize 6656 × 6400 px** (416 × 400 tiles), **1,234 stamps**, 105 patches, 24 roads, 96 anims, 23 animBands, 97 solids, 22 labels, 14 portals, 30 NPCs, 10 vehicles. Every layer reconciles exactly against Σ(fragments) + d00 + merge-injected − 34 pruned.

Runtime is genuinely live `[DONE]`: `?world=full → office-scene.js:464-473` dynamically imports `src/full-world.js`, which fetches `data/city-plan.json?t=<now>`, overrides world size, and calls `renderCityPlan()` — ground pass, per-asset loads, then all layers placed.

A naming trap worth recording: `denizen/src/city/` — the subject of CITY_GENERATOR.md — is **tooling-only**. It reaches just `GET /api/generate-city` and `POST /api/llm-city-plan` plus two tests, and **nothing in the played world imports `cityGenerator.js`**. The two systems share only the word "city."

Build tools. Nine Python scripts: `compile-city-structure.py` (generates d00 from a declarative street lattice), `merge-city-plan.py` (glob → merge → wire teleports → prune → nudge buildings off streets → validate assets exist), `check-city-plan.py` (QA gate, exits 1 on defect), `render-city-preview.py`, `relocate-district.py`, `grid-align-surfaces.py`, `audit-placement-logic.py`, `audit-sprite-frames.py`, `extract-exteriors-full.py`.

Render-preview is Pillow, not a browser — it composites depth-sorted stamps offline with `--crop`/`--scale` for pixel inspection. This is the "build with eyes, never blind" workflow.

⚠ **Process gaps:** `[PARTIAL]`

- **Zero npm wiring.** All 40 `package.json` script entries were checked; there is no `city:*`, `world:*`, or `preview:*`. Every city build step is hand-typed.
- **`merge-city-plan.py` does not call `check-city-plan.py`**, so QA is unenforced. A live run reports **52 OVERLAP + 4 ROADBLOCK = 56 defects, exit 1.**
- **`bbox` has two incompatible shapes** (`{x,y,w,h}` in 12 files, `{x0,y0,x1,y1}` in 4, absent in 2), and `relocate-district.py` transforms every geometry layer but **never updates `bbox`**. Measured drift: `d10-camping.json` has **0 of 51** stamps inside its declared bbox. So the recorded "D3/D12 bbox overlap" is really three *phantom* overlaps of stale metadata, not real stamp collisions.
- **`infra/deploy-full-world-nas.py`'s FILES list omits `data/city-plan.json` and `data/city-specs/` — it ships the blueprint and ~15 MB of assets** but not the city itself.**

Assets. 11,811 PNGs under `denizen/assets/limezu/`. There is **no slicing or atlas for the city** — each stamp names a whole standalone PNG loaded individually, which costs roughly two minutes of load on a dev machine. Three unrelated catalogs coexist (947-object openplan, 1,416 LimeZu, and an

11,803-entry `_asset-catalog.json`). `assets/limezu/index.json` is badly stale (2,334 entries, no `ext` key) but the runtime never reads it.

⚠ **Reproducibility gap:** the untracked `_asset-catalog.json` (11,803 entries, 29 themes) has **no generator anywhere on disk** — a repo-wide grep finds only consumers. The intent is documented ("build it locally with PIL") but the command was never saved, and the file is already stale by 8 PNGs. It is consumed by `infra/setup-city-db-nas.py` to populate `city.limezu_assets` on the NAS.

13 · Security visualization & the Bouncer

The goal is to turn real system and network events into character animations, so someone who cannot read a log file can still see that something bad is happening. A `SecurityMonitorServer` polls system events, network connections and file watches; on Linux, opt-in feeders tail `tshark` and firewall logs. When a threat fires, a robber NPC spawns and performs the matching animation with a speech bubble.

Threat catalog (10 categories): `file_access`, `data_breach`, `network_scan`, `brute_force`, `shell_exec`, `api_abuse`, `process_spawn`, `exfiltration`, `scan_probe`, `packet_anomaly` — each with a detection source and a distinct robber behavior. Dedup drops the same category+source within 5 s; threats auto-resolve after 30 s; at most 5 simultaneous robbers. The Linux feeders are strictly passive — no outbound scanning, no packet injection.

Bouncer is a real investigator. When a threat fires, `src/linux-agent.js` runs live investigation commands inside Kali with a 20-second timeout, and a real terminal window opens so you can watch. `buildReport()` writes a structured Markdown report to `reports/security/` with a verdict of `THREAT CONFIRMED`, `LIKELY FALSE POSITIVE` or `INCONCLUSIVE`. Serious threats become an urgent directive for Dan.

> **Two distinct Kali contexts — do not conflate them.** The **security** Kali is a **WSL distro (`kali-linux`) inside the Windows dev PC** (dolonia, LAN `192.168.1.232`, Tailscale `100.71.54.109`); scans do not run on the physical laptop. The **compute** Kali is the physical laptop `hp81081d` (LAN `192.168.1.240`, Tailscale `100.110.205.49`), a headless mesh worker. A third thing also called bouncer is the mesh job type `bouncer_patrol`, which is TCP/health probing from any fleet node.

The command surface is an allowlist by construction, not by validation. There is no allowlist array, denylist, or validator. Every command is a hardcoded literal keyed by threat category (`src/linux-agent.js:73-136`): `ss`, `netstat`, `nmap -T4 -F --open`, `tcpdump`, `lsof`, `ps aux`,

`who`, `last`, `journalctl`, auth-log greps, `tshark` capture steps, and helpers `whois`, `dig`, `nc -zv`. The only externally-controlled value reaching a shell is an IP, sanitized incidentally by regex capture — `extractIp` turns `1.2.3.4; rm -rf /` into `1.2.3.4` — but with no rejection path. Underneath, `runCommand` is arbitrary `child_process.exec` and is **exported**. No arbitrary-command sink is reachable today; the primitive is one careless call site away.

⚠ Bouncer authentication is fail-open `[BROKEN]`

```
// src/bouncer-investigation.js:273-275
function validateInvestigationAuth(req) {
  const expected = remoteInvestigationToken();
  if (!expected) return true;          // no token configured => everything authorized
```

```
// server.js:3474-3477
function _hasBouncerApiAuth(req) {
  if (bouncerInvestigation.validateInvestigationAuth(req)) return true;   // early return
  const fleet = String(process.env.DENIZEN_FLEET_AGENT_TOKEN || '').trim();
```

The early return means `DENIZEN_FLEET_AGENT_TOKEN` is **never consulted** when `BOUNCER_INVESTIGATION_TOKEN` is unset. The dev PC's `.env` has the fleet token but no Bouncer token, so `/api/bouncer-scan`, `/api/bouncer-linux`, `/api/bouncer-investigate`, `/api/bouncer-patrol` and the agent listening on **`0.0.0.0:8890`** are effectively unauthenticated on that host.

Worse, the WebSocket triggers `bouncer_run_tool` (`server.js:12733`) and `bouncer_run_linux` (`:12812`) reach real WSL command execution with **no auth at all** — the `/agent-ws` upgrade applies only a client-count cap — and neither appears in `MUTATING_CLIENT_TYPES`, so a read-only spectate follower does not block them. Only a per-`category:source` cooldown (default 900 s) throttles them. Token comparison is plain `===`, not constant-time. And `/api/health` reports `investigationConfigured` from the **URL only**, so it reads healthy with no token set.

The shared token lives in three places and can silently diverge: `infra/.bouncer-token.local` (gitignored, read only if the env var is unset); `%APPDATA%\Denizen\bouncer-investigation-token.txt` plus a Windows User-scope env var; and the NAS `.env.production`. Because the setup script sets a User env var, the launcher's env check short-circuits and the local file is never consulted on a logon-triggered run.

Implemented but not deployed: `BOUNCER_MESH_PATROL`, `BOUNCER_FRIDAY_BRIEF`, `ENABLE_TSHARK`, `DENIZEN_INTERNAL_API_TOKEN` are all absent from the NAS env patcher, so mesh patrol never fires in production, the Friday brief is dead code, and the server-side packet feeder is off.

14 · Voice, telemetry & observability

Voice `[DONE]`. Agents speak (ElevenLabs TTS), you speak back (push-to-talk STT), and the office stays silent when you are away. The browser never sees the API key — all TTS goes through `/api/tts`, which adds the `xi-api-key` header server-side. Per-agent voices live in `data/voice-map.json`, falling back to a default voice, then to the browser's Web Speech API offline. A presence gate keeps TTS silent unless you have moved, opened chat, or clicked the canvas in the last 60 s. Push-to-talk STT uses browser speech recognition with a Whisper server-side fallback. An action classifier routes a transcript scored as an action request ("run a security scan") straight to skill execution, closing the voice-in → real-action-out loop. Proximity audio attenuates by distance. ⚠ The 8 voice runtime modules have **zero unit tests**, and `/api/tts` and `/api/stt` are unauthenticated (§3).

LLM telemetry `[DONE]`. Every call is logged as a structured event (id, timestamp, agent, kind, provider, model, latency, tokens in/out, prompt and response previews, error, correlation id). Events live in a 500-entry in-memory ring buffer and append to `data/llm-telemetry.jsonl` — JSONL so partial writes cannot corrupt the file. ⚠ That file is currently **54 MB** and needs rotation. An error classifier turns raw errors into `context_overflow`, `rate_limit`, `auth_failed`, `model_not_found`, `network`, each with a one-line fix hint. Events can forward in real time to Supabase or n8n via an external sink that auto-disables after five consecutive failures. A per-model inference usage tracker (7-day / 50,000-line retention) was added 2026-07-19 and is surfaced at `/llm-usage`.

Surface	How to open	Shows
Status dots	Always visible	Live per-agent state
Work output feed	Tab key	Terminal-style HUD of every agent work output
Diag panel	Bottom-right chip / `GET /api/telemetry`	Queue depth, errors, calls-per-minute, per-agent stats
Proposals panel	Bottom-right chip	Pending animation / SOUL / capability proposals
Mesh jobs panel	Mesh chip / `?mesh=1`	Pending/running jobs, recent completions, fleet CPU/RAM
Security dashboard	Built-in	Live threat detection and robber spawns
LLM usage	`/llm-usage`	Per-model inference usage and cost
Fleet Control Deck	Terminal dashboard	Mesh observability

15 · The compute mesh

Machines report live hardware metrics, register as worker nodes, claim jobs from a shared Postgres-backed queue, execute work including LM Studio inference, and earn contribution credits.

The queue is a Postgres table — `denizen_compute_jobs`, via PostgREST helpers in `src/denizen-compute-queue.js`. The *fleet metrics* registry is separate and **is** in-memory, with 3-minute staleness.

```
enqueue   POST /api/compute/jobs/enqueue    (internal token or fleet identity)
produce   denizen-compute-producer.js       5-min tick: npc_think_batch, bouncer_patrol, grok_te
register  POST /api/compute/node/register   returns endpoint_token; only the SHA-256 hash is sto
heartbeat POST /api/compute/node/heartbeat  desktop every 30s
claim     POST /api/compute/node/claim      optimistic conditional PATCH, not an atomic dequeue
execute   denizen-mesh-job-runner.js        9 job types handled; streaming deltas to /node/progr
complete  POST /api/compute/node/complete   guarded by assigned_node_id + status; then credits
observe   mesh jobs UI + /api/compute/node/status + /api/health fleet block
```

Claiming is optimistic, not atomic. The node reads the top 25 `queued` rows ordered `priority.desc, created_at.asc`, then issues a conditional PATCH `id=eq.X&status=eq.queued → assigned`; losers move to the next candidate. Gated by `jobMatchesNode()` and `nodeCanRunJob()` hardware floors.

⚠ **Failure is terminal — there is no retry and no requeue.** `failJob()` sets `status='failed'`. The only requeue path is `reclaimStaleJobs()`, which flips `assigned` rows older than 30 minutes back to `queued`. Both `expireOldQueuedJobs()` and `trimExcessQueuedJobs()` (`DENIZEN_MESH_MAX_PENDING`, default 8) **fail** jobs rather than retry them.

⚠ "3-node compute mesh" is not what the code supports

There is **no node cap anywhere** — the count is dynamic, being rows in `denizen_compute_nodes`. What exists: the Desktop Electron app and a headless agent script (same script, different `DENIZEN_URL` and `DENIZEN_MACHINE_ID`) which claim all general job types, plus a built-in server node that claims `grok_telemetry_check` **only** and bypasses hardware gates.

In practice, **exactly one worker claims general mesh jobs — the Desktop app on `.232` — and it is running a stale build** (§16). Honest phrasing for external use: *one Postgres-backed queue, an unbounded node model, and in practice a single active general-purpose worker plus a server-side telemetry claimant.*

Node identity is loose. Three layers — `node.id` (UUID PK), `identity_id`, and `machine_id`, which

is **free-form and only ever self-reported**. Auth accepts either `nodeId+token` or `identityId+token`, and the identity path matches the token against **any of that identity's 5 most recent node rows** — so one leaked token authorizes several nodes, and `inference_rpc` targeting relies on an unverified self-declared `machine_id`.

Contribution credits reward completed jobs, uptime windows, host stipends and multi-machine fleet bonuses: base per type (think 5, embedding 8, sim_tick 2, telemetry 1) times hardware/effort/level multipliers, with daily caps (job 300, uptime 80, stipend 50, fleet bonus 10, **total 200**) and a spend catalog offering `priority_boost` and `queue_skip`. Job credits settle inline on completion; uptime and stipends settle on a 15-minute timer to `denizen_credits_ledger`. ⚠ All 10 credits/billing/settlement modules have **zero unit tests** (§17).

21 mesh API paths exist in `denizen-compute-routes.js`, plus 19 `/api/desktop/*` routes; COMPUTE_MESH.md documents 8. `processMeshJob` handles **9 job types** (`npc_think_batch`, `inference_rpc`, `embedding_index`, `sim_tick`, `web_search`, `bouncer_patrol`, `grok_cli_chat`, `claude_cli_chat`, plus mock) and the capability registry declares 11; the doc says 3.

The fake-success bug and its status. `_runWorkerLoop` hardcoded `ok: true` on the completion POST, but `processMeshJob` can *return* failure without throwing — an unsupported job type yields `{ok:false, mocked:true, error:…}` — so no-ops were reported as successes, counted by `incrementJobsCompleted`, and paid credits. An earlier fix addressed **652 fake successes**; `acb3a899` fixed it at the source:

```
const ranOk = !(result && (result.ok === false || result.error));
```

⚠ **The fix is correct and committed but is not running in production** — the installed desktop build predates it and cannot ship until packaging is repaired. Reported failures are still miscounted as successes on the live node.

16 · Clients

Client	What it is	Honest status
Browser (Phaser 3)	The primary pixel-art office. Also a 60-second guided voice tour at `/?demo=tour` and a 20-second investor demo at `/?demo=investor`, plus the `?world=full` city	`[DONE]` — control plane only; the browser cannot contribute compute
Denizen Desktop (Electron)	**v0.1.6**, electron 42.3.3 pinned. Setup wizard, `denizen://` pairing, mesh worker, embedded server spawn, Ollama **and** llama.cpp managers with a brain catalog, auto-updater, and a remote-execution **station agent** (file read/write, whitelisted `node`/`python`/`bash` exec, screen capture, input)	`[BROKEN]` — the installed `app.asar` is dated **2026-06-30**; every rebuild since fails. It is the only general-purpose mesh worker

Headless fleet agent	`scripts/nas-compute-agent.js` for NAS and Kali	`[PARTIAL]` — deployable, but **no systemd unit, so it dies on reboot**
Chrome companion	MV3 extension v1.6.0 — syncs any app or AI workflow with a claimed NPC via selection, clipboard, or `Ctrl+Shift+D`, using a bearer token	`[PARTIAL]` — real and complete for its scope, but **load-unpacked only**, no store listing. Does **no** mesh work
~~VisionClaw (Android)~~	—	**Not a Denizen client.** See below

⚠ **Correction: there are 3 shipped clients and 1 partial, not 4.** `VisionClaw/` is tracked (29 files, Kotlin) but contains **zero** references to Denizen, compute, mesh, or claim. It is a camera/audio WebSocket streamer hardcoded to `ws://192.168.1.232:18791` — and **18791 is OpenClaw's browser-control port**, with `openclaw/` gitignored and zero tracked files. `MONOREPO.md:62` calling it a "Denizen relay" is wrong. Do not list Android as a Denizen client.

⚠ Root cause of the desktop packaging failure `[BROKEN]`

Two prior commits ruled out the asar-integrity fuse, `main.js` bytes, the asar `node_modules` set, the native module, and the electron-builder version, and both said so in their messages. The cause is elsewhere.

`desktop/scripts/copy-denizen-src.js:10-19` holds an 8-file `MODULES` whitelist, **last edited 2026-07-02**. **Two of those eight have since gained** hard top-level requires on modules not in the list**:

- `desktop/src/denizen-inference-runtime.js:2` → `require('./inference-usage-tracker')` — added 2026-07-19
- `desktop/src/denizen-mesh-job-runner.js:6` → `require('./denizen-bouncer-mesh-patrol')` — added 2026-07-07

Both targets exist in `denizen/src/` and are **absent from `denizen/desktop/src/`**. Verified by execution:

```
$ node -e "require('./src/denizen-mesh-job-runner')"
MODULE_NOT_FOUND Cannot find module './inference-usage-tracker'
```

Development survives only because `desktop/denizen-src.js` falls through to a second candidate, `../src` = `denizen/src/`. But **`denizen/src/` is outside the electron-builder `files` root, so it is never in the asar** — which is the entire reason `copy-denizen-src.js` exists. Packaged, both candidates fail, so `main.js:38` → `node-worker-loop.js:10` → `compute-worker.js:5` throws **during main-process

module evaluation.** That matches the symptom exactly (main process starts, no claim loop, no log, no crash dump) and the timeline (last working build 2026-06-30, before both requires landed).

The uncommitted `main.js` diagnostic instrumentation writes breadcrumb "A" *before* the require at `:38`, so **a diag log containing only "A" confirms this diagnosis.** Also missing from the copy list, but soft because they are lazily or defensively required: `lm-studio-router`, `inference-sampling`, `lm-studio-model-manager`, `grok-cli-chat`, `claude-cli-chat`, and `../jelly/paths`.

Dead code to delete: `desktop/compute-job-runner.js` hardcodes `require('../src/…')`, bypassing the shim so it is unfixable when packaged, and nothing references it — yet `COMPUTE_MESH.md:93` still lists it as a key source file.

17 · Repository map, testing & operations

Repository layout

The GitHub project is **DENIZEN**, a monorepo; the application lives in `denizen/`. **18,764 tracked files, 246,487 LOC across 846 `.js/.mjs` files**.

Path	What it is
`denizen/server.js`	13,537-line HTTP + WebSocket + API + orchestrator monolith, no framework
`denizen/src/` (334 top-level / 374 recursive)	Runtime modules — **note ~18 of the 30 largest are browser code**, not server
`denizen/office-scene.js`	8,086-line Phaser office scene
`denizen/jelly/`	Skill runtime, native/vendor skills, sidecar services
`denizen/npcs/<name>/`	`SOUL.md` (commit) + `MEMORY.md` (runtime, do not commit) — 20 directories
`denizen/crm/`	Meridian React SPA + docs (user guide, architecture, competitive analysis)
`denizen/discord-bot/`	**70 files / 20,431 LOC** — the Discord bot, spawned as a child of `server.js`
`denizen/discord-map/`	**8,473-line `game.js`** + 2,174 assets — the Spaces walkable map
`denizen/discord-commands/`	Generated command manifest (130 commands)
`denizen/supabase/migrations/`	**51** ordered SQL migrations — schema, triggers, RLS
`denizen/data/`	184 files — 125 committed canon, **59 runtime-untracked**
`denizen/assets/limezu/`	**11,811 PNGs**
`denizen/desktop/`	Electron app v0.1.6
`denizen/mcps/`	135 files — vendored MCP servers
`denizen/reports/`	Agent-generated Markdown artifacts (security, code-reviews, bugs, dossiers)
`denizen/docs/` (93 docs)	Canonical deep-dives; start at DOCS_INDEX.md
`infra/`	**392 entries**, 330 Python — NAS/Kali deploy, hotfixes, probes. 85 are `_`-prefixed one-offs
`discord pixel game/`	1,607 tracked files — PixelSpaces asset packs

`VisionClaw/`	Kotlin camera/audio streamer — **not a Denizen client**

`denizen-landing-page-interactive/` and `openclaw/` are **gitignored** and should not appear in a repo map. `MONOREPO.md` is stale: it omits `discord pixel game/` and still says sixteen NPCs.

Key source files

File	Responsibility
`src/npc-brains.js` (7,131 lines)	Brain manager — souls/memory, inference queue, `think()`/`getResponse()`, goals, plans, theory of mind
`src/cofounder-agent.js`	CTO director loop — state-reactive prompt, command broadcast, fatigue
`src/world-state.js`	Single in-memory source of truth — positions, fatigue, threats, tasks; pub/sub
`src/agent-bus.js`	Default-deny NPC-to-NPC messaging with buffering and wildcard mirroring
`src/coding-task-runner.js`	Isolated git-worktree coding agent + proposal lifecycle
`src/task-chain.js`	Multi-step chain orchestration with artifact handoff
`src/hallucination-guard.js`	Two-layer grounding check — flags unsupported completion/factual claims
`src/linux-agent.js`	Bouncer's Kali WSL command runner + perimeter patrol
`src/denizen-npc-mcp-gateway.js`	Per-NPC MCP gateway — tokens, scopes, tool dispatch, OpenAPI spec
`src/denizen-compute-routes.js`	Mesh HTTP surface — node/fleet registry, pairing, credits, inference RPC
`src/denizen-crm-routes.js`	CRM HTTP surface — 35 handlers
`src/dossier-system.js`	Branded dossier pipeline — HTML from facts/skins, honesty pass, PDF, proposals
`jelly/index.js`	`createJelly()` runtime entry + skill orchestration

Testing — the real numbers

Denizen uses Node's built-in test runner (`node:test`) — no Jest, no Mocha — via a custom serial wrapper, `scripts/run-tests.mjs`, which spawns one child per file with a 30 s/file timeout.

Location	Files	Executed cases	In `npm test`?
`denizen/tests` (+ `e2e/`)	191	1,438	yes
`denizen/desktop/tests`	1	1	yes
`npm test` set	**192**	**1,439**	—
`denizen/discord-bot/test`	22	120	**no — never invoked by `npm test`**
`denizen/crm/vendor` (vitest 4.1.0)	4	22	no
Total across all three runners	**219**	**1,581**	

Live run, 2026-07-25: `cd denizen && npm test` → 89.9 s, **exit code 1**.

```
[run-tests] files=186/192 tests=1432/1439 failed=7 skipped=0 todo=0 elapsed=89860ms
```

1,432 pass / 7 fail / 6 failing files. All five assertion-failing files reproduce deterministically

standalone — these are not flakes. They are enumerated in

[Appendix G](#appendix-g--open-defect-register).

⚠ **Coverage is 187 of 334 `src` modules = 56%; 147 modules are never touched by any test.** Whole

areas with **zero** coverage:

Area	Modules	Why it matters
Credits / billing / settlement	10	**The money paths have no unit test** — only a smoke that needs a live server
Guardrails / safety	6	`hallucination-guard`, `escalation-engine`, `conflict-detector`, `permission-registry`, `movement-guard`, `freeze-watchdog` — the honesty rail is the brand signature and it is untested
Front-end / browser UI	~35	Only Playwright probes against a live server
Desktop mesh-node runtime	8	And 4 of the 8 are in the current uncommitted diff
Voice / TTS	8	
SAM.gov / market data	5	Despite the feed being live
Screen observation / computer-use	6	
Infra plumbing, research pipeline, CRM peripheral routes	16	
`server.js` (13,537 lines)	—	**No dedicated behavioral test**

⚠ **12% of tests assert on source text, not behavior.** 22 of 191 files use `readFileSync` plus

`assert.match/doesNotMatch` against source strings. They pass on dead code and break on harmless

refactors — and both `discord-integration-security` failures are exactly this pattern, which is why that

failure needs a human to decide whether a guard was dropped or a test went stale.

⚠ **`denizen-crm-integration.test.js` reports PASS with 0 tests** when `SUPABASE_URL` is unset; three

real database round-trip tests silently never execute. It is the only such file.

⚠ **CI does not cover the server.** There is exactly one workflow, `.github/workflows/desktop-release.yml`,

which builds desktop installers. There is **no server CI** — no test run, no lint, no image build, no

deploy gate. Any claim of "CI on every push across Ubuntu + Windows × Node 20 + 22" is not true today.

Also: **43 smoke/probe scripts** live in `denizen/scripts` (18 wired to npm, `smoke:all` chains 15);

31 of 39 require a live server and 13 also require Playwright.

PART II — Product Requirements Document

v1.7.0 → v2.0. Written against the live codebase: every requirement is implemented and marked,

in progress, or planned, with no aspirational fiction.

§1 · Problem, vision, goals & non-goals

Problem. Teams are deploying fleets of autonomous AI agents faster than they can supervise them. The tooling to watch those agents is a wall of logs, JSON traces and dashboards that answer "what happened" but never "what is the shape of the work right now — who is blocked, what is waiting on what, where is the bottleneck." The result is agent systems that silently stall, drift, or make confident-but-wrong claims with no human able to catch it at a glance.

> **Vision.** Make autonomous AI work legible. Denizen turns an agent fleet into a place you can walk
>
> through — a living office where every agent's state, output and blockers are visible in two seconds,
>
> every real action is gated where it matters, and every action quietly compounds into a smarter system.

Goal	Definition of done	Current
Legibility	An observer with no logs open can state within 5 s who is blocked and what is in progress	Met
Trust	Every code- or identity-mutating action passes a human gate; no agent edits its own soul/memory or merges unreviewed	**At risk** — the gates exist, but `soul-proposal/apply` and `assign-directive` are unauthenticated (Part I §3)
Portability	A new vertical stands up by editing soul files and the org chart — no engine changes	Met
Compounding data	Every action becomes a labeled fact/outcome row; accuracy is measured and trends up	**Not met in production** — 4 spine migrations are skipped on the NAS (Part I §7)
Cost control	Reasoning runs cloud, high-frequency and private work runs local; one GPU never melts	Met (serial queue; `LM_STUDIO_PARALLEL=1`)

Non-goals. Not a game — the office is the interface, with no win condition or score. Not a system of record — Denizen links and enriches business truth, it does not replace it. Not an autonomous trader — the market integration is read-only by construction. Not a fully trustless network — the mesh needs ledger replication and open enrollment. Not a no-human-in-the-loop product — approval gates are a feature.

§2 · Target users & personas

Persona	Context	What they need
The Operator — solo founder / small team running an AI-powered company	Runs the office daily as "CEO"; assigns directives; reviews proposals and code; reads the morning briefing	One place to see the whole team's state, approve agent work safely, and have routine business work happen autonomously
The Engineer — evaluating or extending the platform	Reads source, runs locally, swaps models, adds skills and agents	A legible architecture, a documented module and route map, deterministic tests, and safe extension points
The Teammate — collaborator with a claimed	Enters a per-user virtual office; connects their own Claude/Cursor/Custom GPT to a	A private assistant with its own memory, reachable from tools they already use,

assistant	claimed NPC via MCP	without sharing the operator's office
The Discord community operator *(new)*	Runs a guild; wants moderation, engagement and a walkable pixel space; pays per guild	Working moderation and security modules, a map their members can enter, and clear premium value

> **Product judgment baked into the personas.** The shift from a single shared office to per-user virtual offices, and the removal of privacy-leaking labels, were deliberate calls driven by real usage friction. Onboarding was reframed into city/citizenship language and NPC chat into a familiar messaging pattern. The Discord persona is the newest and the only one with billing attached.

Buyers: the AI-forward SMB founder who wants an "AI company in a box"; the agent-platform or dev-tools team that wants the observability layer itself; the regulated or federal operator who wants self-host, RLS, read-only safety rails and the federal RFP pipeline; and **the Discord community operator**, who is the only buyer currently transacting.

Remapping the roster. Because every agent is a soul file, the roster is a template. Documented remappings: customer support (engineers → tier-1/tier-2/escalation, Sarah → feedback prioritization, Bouncer → fraud detection); sales pipeline (Lucy → outbound, Alex → deal desk, Sarah → pricing approval, Bouncer → compliance gate); research lab; trading desk; hospital ops.

§3 · Jobs-to-be-done & user stories

- **JTBD-1.** When my agent fleet is running, know its state at a glance, so I can intervene before something silently stalls.

- **JTBD-2.** When an agent wants to change real code or its own identity, approve it first, so autonomy never becomes unaccountable.

- **JTBD-3.** When routine business work arrives, have agents handle it within guardrails, so I focus on judgment calls.

- **JTBD-4.** When I onboard a teammate, give them a private assistant reachable from their own tools, without exposing my office.

- **JTBD-5.** As the system runs, have it learn — predict outcomes and get more accurate — so the data becomes an asset.

- **JTBD-6.** *(new)* When I run a community, give members a shared space and give me moderation that works, at a price per server.

ID	Story	Status
US-01	See colored status dots and task labels above every agent, updated twice a second	Done
US-02	Press Tab to read a terminal-style feed of every agent's work output	Done
US-03	Assign a direct-order task over chat or REST and watch it report back	Done ⚠ unauthenticated

US-04	Review and approve/reject every agent code change before it merges	Done
US-05	Triage inbound email, auto-send safe replies, draft anything hitting an escalation keyword	Done
US-06	Connect a Custom GPT / Cursor / Claude to a claimed assistant via bearer-token MCP	Done — 38 tools
US-07	Get a morning briefing from project state, handoffs, commitments, pipeline and top predictions	Done ⚠ pipeline is cross-org
US-08	Manage contacts, deals, sequences, quotes and a client portal inside the office	Done
US-09	Land on a public demo and watch a small office work, read-only, with no setup	**Done** — `DENIZEN_PUBLIC_DEMO=true` is live
US-10	See each agent's work scored 0–5 and a team competency map driving training directives	Planned (5.x)
US-11	*(new)* Pay per guild to unlock security and community modules	**Done** — Stripe live
US-12	*(new)* Walk a pixel map tied to my Discord identity	Done
US-13	*(new)* Contribute compute from my desktop and earn credits	**Blocked** — packaging (Part I §16)

§4 · Functional requirements

Observability & office

Pri	Requirement	Status
P0	Live status dot (8 states) + task label per agent, ≤ 500 ms	Done
P0	Terminal-style work-output feed (Tab toggle)	Done
P0	Talk to any agent as the operator; in-character replies with memory	Done
P1	Diag panel: queue depth, error classes, calls-per-minute, per-agent stats	Done
P1	Per-model LLM usage and cost tracking	Done (2026-07-19)
P2	Public, read-only demo office	Done

Agent intelligence

Pri	Requirement	Status
P0	Per-agent think loop, 22-section context, strict JSON decision contract	Done
P0	Persistent per-agent memory with skill/topic/outcome tags	Done
P0	Director loop: 2–4 commands/cycle, scripted fallback after 5 errors	Done
P1	Mood system biasing decisions	Done
P1	Hallucination guard: flag unsupported claims, inject "verify or retract"	**`[PARTIAL]` — advisory only, and its caller is OFF by default (`DENIZEN_REAL_WORK_MODE=false`); untested**

P1	Daily SOUL reflection proposes ≤ 1 identity edit, operator-gated	**`[PARTIAL]` — review/apply live, but `reflectOnDay` has zero callers**
P1	Centralized sampling profiles per task kind	Done
P1	Semantic / vector memory retrieval	**Not built** — top-k keyword overlap only; no vector store exists
P1	Grok/Claude mentor loop feeding lessons back into memory	`[PARTIAL]` — false-failure loop fixed 2026-07-22

Capabilities & safety gates

Pri	Requirement	Status
P0	Skill bridge via Jelly	Done
P0	Coding agent in an isolated worktree; diff becomes an approval-gated proposal; forbidden-file guard	Done
P1	Cross-agent task chains with artifact handoff; conflict detection → arbitration	Done
P0	No self-modification of soul/memory/settings; no unreviewed merge; no order placement	**At risk** — enforced in the guard, but the apply endpoint is open
P0	**Authenticate mutating routes**	**Not met** — 237/372 sites unguarded

Business surfaces

Pri	Requirement	Status
P0	Meridian CRM: contacts/deals/pipeline/sequences/quotes/portal/analytics with cloud sync	Done
P1	Lucy email triage with auto-send + escalation-keyword draft-only	Done
P1	Federal RFP pipeline: status rollups, requirement readiness, bid probability	`[PARTIAL]` — no org filter; `rfp_requirements` never written
P1	SAM.gov opportunity feed → RFP pipeline → CRM deal mirror	Done (nightly, on by default)
P1	Read-only market watch + trade suggestions; Robinhood OAuth	OAuth needs 1× login
P2	AI Prospector	**UI-only**
P2	Marketing campaign send	**UI-only** (persists, does not send)

Multi-tenant, identity & mesh

Pri	Requirement	Status
P0	Org-scoped RLS on tenant tables; protected agents un-claimable; demo org walled off	`[PARTIAL]` — 37/77 tables have RLS; service role bypasses
P1	Per-NPC MCP gateway + OpenAPI for Custom GPT	Done — 38 tools, 6 scopes, audited
P1	Team MCP gateway with `run_as_npc`	Done ⚠ no rate limit
P1	Per-user virtual offices: isolated NPC team + memory	Done
P1	Compute mesh: enqueue/claim/complete + credits ledger	`[PARTIAL]` — one active worker; no retry
P2	Apply production migrations to the remote instance	**Built, 47/51 applied**

Discord *(new section)*

Pri	Requirement	Status
P0	Bot with moderation, logging, automod	Done — 130 commands
P0	Per-guild Stripe plans gating premium modules	Done
P0	Walkable Spaces map tied to Discord identity	Done
P1	Seller slots (SlotBot parity)	Done
P1	Credits economy, shop, PVP duels, minigames	Done
P1	Music (Lavalink)	`[BROKEN]` — voice dep missing, false success
P1	Avatar closet with plan-gated cosmetics	Done
P2	Free-plan watermark	**Server computes it, client ignores it**
P2	Slash-command budget management	`[BROKEN]` — 111 vs 100 cap, trim env unset

§5 · Non-functional requirements

Category	Requirement	Current
Performance	One local GPU serves the roster without OOM; ~one think every 3–4 s office-wide	Met (serial queue)
Performance	Status indicators ≤ 500 ms; UI responsive during inference	Met
Performance	Prompt-token reduction + memory auto-summarization + chat-queue priority	Ongoing
Reliability	Office never goes silent: provider → Claude → scripted fallback	Met
Reliability	Idempotent restarts via `office_session_id`; mesh watchdog	Met
Reliability	Mesh jobs retry on failure	**Not met** — failure is terminal
Security	Org-scoped RLS; bearer tokens as SHA-256 hashes; secrets never sent to the browser	`[PARTIAL]` — hashing and browser isolation met; RLS coverage 37/77; 64% of routes unguarded
Security	Authenticated security-tooling endpoints	**Not met** — fail-open (Part I §13)
Safety	Read-only trading; draft-only sensitive email; forbidden-file guard; approval gates	Met in the guards
Privacy	Local models for code reads and high-frequency work; per-user office isolation; GDPR export	Met
Portability	Identity = Markdown; org chart = data	Met
Observability	Structured telemetry (ring buffer + JSONL), error classifier, health endpoint	Met ⚠ no external polling, no alerting, no log rotation
Deployability	Reproducible container config	**Not met** — the deploy script writes an env file nothing loads (Part III M4)
Recoverability	Database backup	**Not met** — none exists (Part III M10)
Testability	Deterministic suite on the built-in runner	`[PARTIAL]` — 1,432/1,439 pass; 56% module coverage; **no server CI**

§6 · Success metrics & KPIs

Metric	Definition	Target
Glance-to-truth time	Seconds for an observer to state who is blocked / in progress	< 5 s
Prediction accuracy	Validated / total scored predictions	**Re-baseline required** — see Part I §7
Autonomous action rate	Share of decisions that execute a real skill vs idle/talk	Rising without error spike
Approval-gate coverage	Code/identity mutations passing a human gate	100% — **currently gapped by open routes**
Hallucination flags resolved	Unsupported claims caught and corrected	Trend down per agent
Office uptime	Time the office is "alive" including all-AI-offline fallback	≥ 99%
Test pass rate	Suite green	**Currently 1,432/1,439 — target 100%**
Paid guilds *(new)*	Active Stripe subscriptions	Operator-tracked

Business metrics: activation (time-to-first-claimed-assistant); weekly active offices and operator retention; agent-hours of autonomous work per week; CRM pipeline value touched by agents; mesh credits earned and spent; **and, new, paid guild conversion and retention** — the only surface currently producing revenue signal.

> **North-star metric.** Autonomous agent-hours per supervised hour. Every other metric serves this ratio — how much real work the agent team produces per hour a human spends supervising it, without breaching a safety gate.

§7 · Release plan & milestones

Phase	Theme	Status
Foundation	Roster, movement, memory, soul files, code awareness, artifact output, task cascade	Done
Phase 1	Mood system, directive inbox, skill bridge	Done
Phase 2	Lucy email + 7-step job pipeline, Bouncer investigations + Dan escalation, work feed	Done
Phase 3	Agents run Claude Code on real files, task chains, conflict detection → arbitration	Done
Compute mesh	Shared job queue, fleet metrics, contributor nodes, credits	`[PARTIAL]` — built; one active worker
Meridian CRM	Full embedded CRM + support, marketing, team modules	Done (2 surfaces UI-only)
Phase 4.1	Public demo mode	**Done** — live in production
Phase 4.2	Performance — token reduction, memory summarization, chat priority	Ongoing

Phase 4.3	Docker production deploy — env passthrough, HEALTHCHECK	`[PARTIAL]` — healthcheck done; **env passthrough broken**
Discord product	Bot, Spaces map, per-guild Stripe tier, seller slots, credits economy	**Done (new)**
SAM.gov feed	Federal opportunity ingest → RFP pipeline → CRM mirror	**Done (new)**
City / full world	13 districts, road lattice, enterable shops, `?world=full`	**Done (new)**, QA unenforced
Intelligence spine in production	Apply the 4 skipped migrations	**Not started** — top P0
Phase 5.1	Outcome scoring (0–5 per artifact) + team competency map	Not built
Phase 5.2	Gap-aware reflection + training-task generator	Not built
Phase 5.3	Autonomous capability proposals + trial-mode SOUL edits	Not built

Immediate path to a defensible public launch

The order has changed since v1.6.0. Correctness and safety now precede new capability:

1. **Close the auth surface** — authenticate `soul-proposal/apply`, `assign-directive`, `grant-permission`, the market writes and the telemetry reads; fix the Bouncer fail-open path and the two unauthenticated WS command triggers.
2. **Contain the data-exposure items** — the untracked live-CRM report file, and the cross-org federal pipeline read.
3. **Make production reproducible** — point the deploy script at `.env.production`, then apply the 4 skipped spine migrations so the learning loop is actually closed.
4. **Add a database backup** and container log rotation.
5. **Unbreak the desktop build** (one-line fix to the copy whitelist), which also ships the mesh fake-success fix and restores the only general-purpose worker.
6. **Get the suite green** and add server CI so it stays green.
7. **Then** Phase 5.1 — the scoring loop that turns "agents doing tasks" into "a team learning to do tasks better."

> **What Phase 5 unlocks.** Work is scored, scores propagate to agents as skill points and mood, a team competency map updates, and Abby generates training directives targeted at weak areas — all behind the same non-negotiable approval gate. That is the difference between an agent demo and a system with a compounding moat. It is also why item 3 above matters: without the spine closed in production, there is nothing to score against.

§8 · Risks, mitigations & open questions

Risk	Mitigation in place / planned
Single-GPU saturation	Serialized queue; local fast tier; `LM_STUDIO_PARALLEL` tunable; mesh offload
Agent hallucination	⚠ **Largely unmitigated in practice.** The evidence check is advisory, its caller is disabled by default, the model verifier is dead code, the flag event has no consumer, and the guard has no tests. Three fake-success bugs shipped in this period, all caught by the operator rather than the guard
Identity drift	SOUL edits ≤ 1/day behind a human gate ⚠ **the apply endpoint is unauthenticated, and the proposal generator is never called**
Chain/state loss	⚠ **None** — task chains are in-memory only and lost on every restart
Multi-instance NPC messaging	⚠ **None** — the agent bus is an in-process EventEmitter; multi-instance deploys silo NPC↔NPC messaging entirely
Unaccountable autonomy	Forbidden-file guard, approval-gated merges, read-only trading, draft-only sensitive email ⚠ **undermined by the open route surface**
Multi-tenant data leakage	Org-scoped RLS; protected agents un-claimable; demo org walled off ⚠ **37/77 tables; federal pipeline reads cross-org**
Data loss	**No mitigation — there is no database backup**
Non-reproducible production config	**No mitigation — the deploy script writes an unread env file; live config is hand-accumulated**
Cloud provider cost/latency	Per-role provider split; local models; fallback chain; per-model usage tracker
Operational fragility (home lab)	Watchdogs, PM2 (⚠ **broken**), idempotent migrations, health endpoint ⚠ **AtLogOn scheduled tasks die on logoff; no NAS cron**
Licensing exposure	**Unresolved — non-commercial-labeled art in a paid Discord SKU**
Solo-maintainer bus factor	93 docs, a canonical index, deterministic tests, and this dossier ⚠ **but 147 modules are untested and CI does not cover the server**

Open questions. Hosting model — self-host-first (regulated/federal) versus managed cloud (SMB)? Demo scope — how much does the read-only demo expose before it becomes a support burden? Mesh trust — minimum viable path to ledger replication and open enrollment? Pricing unit — per-seat, per-office, per-agent-hour, per-mesh-credit, or (as Discord now proves) **per-guild**? CRM positioning — wedge feature, standalone product, or strictly the business surface for the agent team? And newly: **is Discord the actual wedge?** It is the only surface with paying customers, and it was not in the strategy document a month ago.

PART III — Technician & Maintenance Manual

Everything needed to install, configure, operate and repair a Denizen deployment. Written for the person on call at 2 a.m. Commands assume the application root at `denizen/`.

M1 · System requirements

Component	Requirement
Runtime	Node.js 22+ (verified on v22.13.1)
OS	Windows, macOS or Linux for the server; WSL2 with a Kali distro for Bouncer's live investigations
GPU (local models)	NVIDIA GPU for LM Studio; reference deployment uses an RTX 4070 Ti Super
Local model host	LM Studio serving an OpenAI-compatible endpoint on `localhost:1234`; Ollama optionally probed in parallel
Database	Supabase / Postgres (self-hosted on the NAS in production)
Optional	ElevenLabs (TTS), n8n (email/automation), Tailscale, Docker, Redis (world pub/sub), **Java Lavalink v4** (Discord music)

M2 · Install & first run

```
git clone https://github.com/Dolonia333/DENIZEN.git

cd DENIZEN/denizen

npm install

npm start            # node --env-file=.env server.js

# open http://localhost:8080
```

> **Zero-config first boot — verified true.** No environment variable is required to start. There is no
>
> `process.exit(1)` or module-load throw on missing config; the database self-disables with a log line.
>
> **Caveat worth stating plainly:** zero-config yields *scripted-fallback* NPCs, not live agents. Add
>
> provider keys to switch to real inference.

M3 · Configuration

There are 455 distinct environment variables read by `denizen/`, across three access patterns — `process.env.NAME` (392), `env.NAME` passed as a parameter (+25 unique), and `envBool('NAME', …)` with the name as a string literal (+38 unique). A `process.env.` grep alone **undercounts by 63.** Of the 441 that are application config, roughly **57 are real secrets**.

The most-used:

Variable	Purpose
`PORT`	HTTP/WS port. **Defaults disagree: `server.js` 8080, desktop 8081, production compose 8888**

`ANTHROPIC_API_KEY` / `XAI_API_KEY` / `KIMI_API_KEY` / `MOONSHOT_API_KEY`	Cloud provider credentials
`LM_STUDIO_MODEL` / `_HEAVY_MODEL` / `_FAST_MODEL`	Pin the three local tiers
`LM_STUDIO_PARALLEL`	Interactive inflight cap, clamped 1–8, **default and live value 1**
`LM_STUDIO_HEAVY_TTL_SEC`	JIT unload for the heavy tier (default 600)
`SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY`	Spine + CRM workspace. ⚠ Service role **bypasses RLS**
`DENIZEN_DB_ENABLED`	Master DB switch
`EMAIL_AUTOSEND` / `EMAIL_INBOUND_SECRET`	Lucy auto-send; n8n shared secret (else 401)
`ELEVENLABS_API_KEY`	Server-side TTS; never exposed to the browser
`SAM_GOV_API_KEY` / `SAM_GOV_AUTO_INGEST`	Federal feed; nightly ingest is **on by default**
`DISCORD_BOT_TOKEN` / `DENIZEN_DISCORD_BOT`	Bot credentials; `off` disables the child process
`DISCORD_SLASH_TRIM`	**Must be set — unset changes which 11 slash commands are dropped**
`STRIPE_SECRET_KEY` / `STRIPE_PRICE_PRO` / `_ULTRA` / `_PET` / `STRIPE_GUILD_WEBHOOK_SECRET`	Discord billing
`BOUNCER_INVESTIGATION_TOKEN`	⚠ **Unset means fail-open — set it**
`DENIZEN_MAX_WS_CLIENTS`	WS cap (50 default, 30 live)
`DENIZEN_PUBLIC_DEMO`	Public demo mode — **true in production**
`MIROFISH_ENABLED`	Off unless set; heuristics are the only path otherwise

Dead configuration to prune: `GEMINI_API_KEY` (documented in `PRODUCTION_DEPLOY.md` but read by no server code — there is no Gemini provider), `DENIZEN_HOST_ON_NAS`, and `LAVALINK_AUTO` (the real switch is the inverted `LAVALINK_DISABLE`).

M4 · Service topology & ports

Authoritative for production: `denizen/docker-compose.production.yml`, `network_mode: host`, `restart: unless-stopped`, `PORT: ${PORT:-8888}`.

Service	Host / role	Port	How it starts	Status
denizen-server	NAS `192.168.1.237`, Docker, host network	**8888**	`docker compose up -d`	Live primary
denizen-follower / -b	NAS, same host	8889 / 8890	`docker run -e DENIZEN_SERVER_ROLE=follower`	Conditional (Redis pub/sub)
Discord bot	**child process inside denizen-server**	5180 internal	`spawn(...discord-bot/index.js)`	Live when token set — **not a container**
supabase-db	NAS Docker	5432 / Kong 8000	Supabase self-host compose	Live
denizen-lavalink	NAS Docker	2333	`_setup_lavalink_nas.py`	Live

	denizen-n8n	NAS Docker	5679 → `n8n.dolonia.cloud`	`provision-n8n-tunnel.py`	Live (Angela Gmail → CRM)
	denizen-redis	NAS Docker	6379, **bound 127.0.0.1**	`provision-redis-nas.py`	Conditional
	LM Studio	dolonia `192.168.1.232`	1234	**Manual GUI launch**	Live — **single point of failure**
	Bouncer investigation agent	dolonia (WSL `kali-linux`)	**8890, bound 0.0.0.0**	Windows Task, trigger **AtLogOn**	**Dies on logoff**
	Kali compute agent	`hp81081d` `192.168.1.240`	—	Hand-added `@reboot` crontab	Fragile — no systemd unit
	Desktop mesh node	dolonia (Electron)	8081	Windows Task, AtLogOn	**Broken build** (Part I §16)
	cloudflared	NAS Docker	—	Pre-existing tunnel	**The only ingress**
	Tailscale	dolonia + Kali	—	OS service	**The NAS is not on the tailnet**
	Reverse proxy	**None for the app** (Kong fronts Supabase only)	—	—	TLS terminates at the Cloudflare edge
	WebSocket — office	`/agent-ws`	—	—	Agent thought, player chat, commands
	WebSocket — security	`/security-ws`	—	—	Threat broadcast + robber spawns
	SSE — CRM	`GET /api/crm/workspace/stream`	—	—	Workspace push
	SSE — inference	`GET /api/compute/inference/stream`	—	—	Mesh inference stream
	Health	`GET /api/health`	—	—	Process, ws, visitor.db, fleet, bouncer blocks
	PM2 (`ecosystem.config.js`)	—	8080	—	⚠ **Dead and**

				broken — see below

⚠ **Do not use `pm2 start ecosystem.config.js`.** That file is not referenced by `package.json` or the production Dockerfile, and line 11 calls `.splitlines()` — a **Python** method. It raises a `TypeError` at runtime and would crash whenever `.env.production` exists. Earlier versions of this dossier prescribed it in both the routine-operations section and the command quick-reference; that guidance was wrong.

⚠ The deploy script writes an env file nothing loads

`deploy-denizen-to-nas.py:355,401` upload the generated bundle to `{REMOTE_DIR}/.env`, but `docker-compose.production.yml:8` declares `env_file: - .env.production`. Plain `.env` is only used by compose for `${PORT}` interpolation. `patch-denizen-nas-env.py:15-19` carries an in-code post-mortem of exactly this: *"Patching plain .env wrote to a file nothing loads — these patches silently had no effect (this is why the CLI bridge env never reached the server). Also: docker restart does NOT reload env."* The fix landed for the `patch-*` family but **not for the deploy script.** Consequence: **there is no reproducible source for the live container environment** — it is whatever the `patch-*` scripts have accumulated by hand. Two stragglers still write plain `.env`, and about ten more *read* it to recover tokens, so they may be reading stale values. Separately, `docker cp` hotfixes write to the container's writable layer, which any `compose up --force-recreate` destroys.

M5 · Routine operations

```
# foreground
npm start


# containerized (production)
docker compose -f docker-compose.production.yml up -d
docker logs --tail 200 denizen-server
```

Process watchdog (Windows host). The repo ships `watchdog.ps1` / `watchdog.bat` and `check-status.ps1` to keep the host process and LM Studio alive. Run the watchdog as a scheduled task on the GPU host so an inference crash self-recovers.

Smoke tests.

```
npm run smoke:mesh          # mesh register/claim/complete (13 checks)
npm run smoke:mesh-jobs     # enqueue -> progress (5 checks)
npm run smoke:nas           # NAS fleet reachability (5 checks, not 4)
npm run smoke:credits       # credit economy
npm run smoke:spine         # facts -> outcomes -> predictions e2e
npm run smoke:all           # the full chained suite (15 scripts)
```

⚠ 31 of 39 smoke scripts require a live server and 13 also require Playwright.

Tests — remember there are three runners.

```
cd denizen            && npm test   # 192 files / 1,439 tests
cd denizen/discord-bot && npm test   # 22 files / 120 tests -- NOT in the main suite
cd denizen/crm/vendor  && npm test   # vitest, 4 files / 22 tests
```

> **Operating cadence.** On the GPU host, keep LM Studio and the Denizen process under the watchdog. On the NAS and Kali nodes, keep the compute agent under their own supervisors. Confirm a green `/api/health` fleet block before assigning heavy directives or task chains. Note that both Windows scheduled tasks are **AtLogOn** — they are down whenever nobody is logged into the dev PC.

M6 · Database operations

```
py infra/apply-migrations-nas.py      # idempotent, additive-only; safe to re-run
py infra/verify-migration.py          # confirm applied set
py infra/compare-schema.py            # diff local definitions vs the remote instance
```

⚠ **The runner deliberately skips four migrations.** `SKIP_VERSIONS = {20260606000005, 000006, 000012, 000013}` — they need the Dolonia platform CMS tables. **Production therefore runs 47 of 51**, and lacks the outcome triggers, the funnel/velocity views and the platform identity bridge. Check `GET /api/platform-bridge/status`. Closing this is the top P0 in Part II §7.

Never edit an applied migration; add a new one. To deprecate a column: stop writing it → migrate readers → drop in a later migration. Watch the three chain defects in Part I §8 — in particular the `20260707000001` version collision, where one of two migrations is silently skipped.

M7 · Compute-mesh operations

```
py infra/finish-all-mesh.py     # provision NAS + Kali bundles, apply migrations, deploy agent,

py infra/probe-nas-status.py    # NAS services up?

py infra/probe-nas-agent.py     # compute agent reachable?

py infra/probe-cloudflared.py   # public tunnel healthy?

npm run drain:mesh              # drain the queue before maintenance
```

Tailscale repair for an offline Kali node: `infra/repair-tailscale.ps1` / `.cmd`, then re-probe.

Credits settle on a 15-minute timer; verify with `npm run smoke:credits`.

⚠ Remember that mesh job failure is terminal — inspect `status='failed'` rows directly; only `reclaimStaleJobs()` requeues, and only for `assigned` rows older than 30 minutes.

M8 · Monitoring & observability

Signal	Where	Watch for
Inference queue depth	Diag panel / `GET /api/telemetry`	Sustained growth = GPU saturation; reduce parallelism or offload
Error classes	Telemetry ring buffer + `data/llm-telemetry.jsonl`	`context_overflow`, `rate_limit`, `auth_failed`, `model_not_found`, `network`
Fleet health	`GET /api/health` fleet block + mesh jobs panel	Offline nodes, stalled jobs, **failed jobs that were reported as complete**
Agent state	Status dots + work-output feed (Tab)	Agents stuck red/idle; clustering
Security	Security dashboard + `reports/security/`	Repeated `THREAT CONFIRMED` verdicts
Model spend	`/llm-usage`	Per-model cost drift

⚠ **`GET /api/health` is the only monitor, and nothing polls it from outside.** No Prometheus, no Grafana, no uptime service, no alerting. The Docker healthcheck probes it every 30 s and `restart: unless-stopped` restarts on crash — that is the whole of automated recovery.

⚠ **Container logs have no rotation.** Neither compose file declares `logging:`, `max-size` or `max-file`, so NAS container logs grow unbounded. Only the inference usage tracker has retention (7 days / 50,000 lines). `data/llm-telemetry.jsonl` is currently 54 MB and must be rotated manually.

⚠ `/api/health` reports `investigationConfigured` from the URL only, so it reads healthy with no Bouncer token set.

M9 · Troubleshooting runbook

Symptom	Likely cause	Fix
All agents default to "work"; speech is generic	JSON parse failures from truncated output, or all providers down → scripted fallback	Confirm provider keys / LM Studio up; ensure `max_tokens` not lowered; check telemetry for `auth_failed` / `model_not_found`
Inference times out; queue depth climbs	Single GPU saturated, or model too large for VRAM	Keep `LM_STUDIO_PARALLEL=1` for 14B+; move heavy agents to the fast tier; offload think batches
Telemetry shows `context_overflow`	Prompt + memory exceed the context window	Run the `lm-studio` skill to swap context size; rely on the 200-line memory cap
`main_model_unloaded`	Pinned `LM_STUDIO_MODEL` does not match a loaded model	Reconcile `.env` with what LM Studio actually has loaded — the docs and live `.env` currently disagree
An agent is stuck red / idle	Pathfinding gave up, or a blocked task with no escalation	Pathfinding reroutes 5× then idles after 8 s by design; check the directive inbox and `[OUTCOME:blocked]`
Mesh node shows offline	Tailscale dropped, agent died, or NAS service down	`repair-tailscale`, re-run the probes, restart the node's agent
Mesh reports successes but nothing happened	The desktop node is running the pre-`acb3a899` build	**Cannot be fixed without repairing the desktop build** — see below
Desktop app starts but never claims jobs; no log	`copy-denizen-src.js` whitelist is missing two transitive requires	Add `inference-usage-tracker` and `denizen-bouncer-mesh-patrol` to `MODULES`, rebuild. Confirm first: `~/denizen-desktop-diag.log` containing only breadcrumb "A"
Roki has live quotes but no portfolio	Robinhood OAuth not completed	Visit `/api/market/auth/connect` once and approve. The system never trades regardless
Lucy sends nothing / everything drafts	`EMAIL_AUTOSEND` unset, or an escalation keyword matched	Set `EMAIL_AUTOSEND=true`; escalation keywords always force draft — by design
n8n email webhook returns 401	`EMAIL_INBOUND_SECRET` mismatch	Ensure n8n appends `?secret=...` matching the env value
Bouncer investigation does nothing	WSL Kali distro missing or misnamed	Confirm WSL `kali-linux` on the GPU host, separate from the physical Kali laptop. ⚠ Also confirm the task is running — its trigger is AtLogOn
A code proposal won't merge	Awaiting approval, > 5 files, or a forbidden path	Approve via the API; split large tasks
CRM edits not persisting across devices	Cloud sync blocked (Supabase creds) — local `meridian_*` only	Set `SUPABASE_*` so `PUT /api/crm/workspace` succeeds
CRM edits vanish after a conflict	**By design** — on `ifUpdatedAt` conflict the client discards local edits and applies server state	Re-enter the change; watch for the 2.5 s "conflict" flash
`,play` says "Now playing" but there is no audio	No voice encryption dep installed; the catch path returns `{ok:true}`	Install `sodium-native` (or `libsodium-wrappers` / `@noble/ciphers`) in `discord-bot`
A slash command vanished after a sync	`DISCORD_SLASH_TRIM` unset → different 11 commands dropped by tiebreak	Set `DISCORD_SLASH_TRIM` explicitly; prefix commands still work
Discord premium denied for a paying guild	Supabase unreachable and the 60 s cache expired	Cache keeps last-known-good on a *failed* refresh, but a cold start with no DB falls back to free
Every Discord map is wide open	`!guildOrg.dbReady()` → allow	Restore Supabase connectivity; the gate fails **open** by design here

Hosted multi-tenant behaves oddly	The 4 skipped spine migrations	Apply them once the platform CMS tables exist
`generate-city-endpoint.test.js` hangs the suite	Its `after()` hook does not reap the spawned server on Windows	Known; costs a hard 30 s and one file failure per run

M10 · Backup, recovery & data hygiene

⚠ **There is no database backup.** `pg_dump` appears exactly twice in the repository, both one-time migration tooling, and one of those dumps the *desktop* database rather than production. **No scheduled, recurring or automated dump of NAS Postgres exists.** The only things named "backup" are single-file config copies taken before a patch.

Data at risk with no repo-visible recovery path: the CRM and GovCon pipeline, SAM.gov opportunities, Dolonia conversation data, the n8n workflow database, the `city` schema, and the `./data` and `./layouts` bind mounts. Synology Hyper Backup may cover the volumes, but **nothing in the repository configures or verifies it** — confirm directly.

What *is* covered:

- **Canon (git).** SOUL files, company brief, catalogs, sprite maps, city specs, source and migrations are version-controlled. Commit SOUL changes after approving a reflection proposal so identity evolution is tracked.

- **Runtime (Postgres).** Facts, outcomes, predictions, CRM workspace, telemetry and visitor data live in Postgres — back up at the database layer. Event-sourced tables are append-only and replayable.

- **CRM.** Use the built-in full JSON backup plus per-entity CSV export; JSON restore supports merge or replace. Per-contact GDPR export covers data-subject requests.

- **Untracked runtime files.** ⚠ The 59 untracked JSON/JSONL files in `denizen/data/` — including the entire Grok/mentor learning loop — are in **neither** git nor Postgres. They are currently backed up by nothing.

- **Local hygiene.** Rotate `data/llm-telemetry.jsonl` (54 MB today); the in-memory ring self-caps at 500 entries. Temp git worktrees from the coding agent are cleaned on approve/reject — sweep orphans after a crash. `MEMORY.md` self-trims at 200 lines; to reset an agent's episodic memory, clear its `MEMORY.md` and leave `SOUL.md` untouched.

M11 · Security & safety-control maintenance

Control	Upkeep

Org RLS	After any new tenant table, confirm `org_id` and a policy using `denizen_user_can_access_org()`; re-run schema verification. ⚠ Audit the 40 tables currently without RLS and record which exemptions are deliberate
Route auth	⚠ **Audit the 237 unguarded dispatch sites.** Start with `soul-proposal/apply`, `assign-directive`, `grant-permission`, market writes, telemetry reads
Bouncer auth	⚠ **Set `BOUNCER_INVESTIGATION_TOKEN` everywhere, fix the fail-open early return, and add auth to the `bouncer_run_tool` / `bouncer_run_linux` WS triggers.** Keep the three token copies in sync; prefer constant-time comparison
Protected agents	Keep `abby`/`bouncer`/`lucy` flagged `protected = true`; they must stay un-claimable and token-ineligible
MCP tokens	Stored as SHA-256 hashes; rotate by revoking and re-issuing (plaintext shown once). ⚠ Note 1-click OAuth mints `full` scope, and "read-only" tokens can write memory
Coding guard	Verify the forbidden-file list still covers `npcs/`, `.claude/`, user-profile and company-brief after refactors
Email safety	Review the escalation-keyword list; keep `EMAIL_INBOUND_SECRET` set in production
Trading safety	Confirm no order-placement path exists; suggestions remain text-only
Security feeders	Keep Linux `tshark`/firewall feeders passive — no outbound scanning, no packet injection
Stripe webhooks	Verify HMAC tolerance and that `stripe_subscription_id` remains uniquely indexed
Repo hygiene	⚠ Confirm no live-customer data sits untracked-but-unignored in `infra/` — see Appendix G item 1

M12 · Upgrade procedure & maintenance schedule

Upgrade. `git pull`; review `CHANGELOG.md` and any new migrations. `npm install` for new dependencies. Apply new migrations, then verify. Run `npm test` **and both secondary suites**, plus `npm run smoke:all`, before re-exposing publicly. For the desktop app, electron-builder publishes auto-updates via GitHub releases — ⚠ and see Part I §16 before attempting a build.

Cadence	Task
Daily	Glance the office and diag panel; clear pending proposals and code reviews; skim `reports/security/` verdicts; confirm a green `/api/health`
Weekly	**Rotate `llm-telemetry.jsonl`**; run `smoke:all`; review the prediction-accuracy trend; reconcile mesh credits; back up the CRM workspace JSON; **check `docker logs` size**
Monthly	**Database backup verification and a restore drill** (⚠ once a backup exists); dependency and security updates; schema diff; review escalation-keyword and forbidden-file lists; prune orphaned worktrees; **re-audit the unguarded route list**; **reconcile the three Bouncer token copies**
Per release	Re-run the [verification appendix](#appendix-d--verification-appendix) and update this document's numbers plus `data/canonical/denizen-facts.json`

PART IV — Builder Profile & Signal

A system is also a statement about the person who built it. This part summarizes an independent build-retrospective and reads it the way an investor would — as evidence about execution capacity and risk. **Figures are reproduced as reported from that retrospective; they are not re-derived here, and they describe the period up to v1.6.0.**

§A · The build, by the numbers

Metric	Value
Total build time	77 hours across 18 distinct working sessions
Longest single session	16 h 0 m
Parallel agents	As many as 2 main sessions at once
Words per prompt	81 — mostly conversational direction, not rigid specs
Sessions referencing product decisions	50%
Builder archetype	"Product Thinker"

§B · What the retrospective found

User-backwards product judgment. The work consistently moved toward the user's mental model rather than the system's. Onboarding was reframed into city/citizenship language; NPC chat became a familiar messaging pattern; privacy-leaking labels were removed; the architecture shifted from a single shared office to per-user virtual offices. These are product calls, not code chores.

Supervision of AI output, not passive acceptance. The retrospective notes catching a project-context problem, challenging a CRM fix that could not be visually verified, correcting a stale assumption an agent made about a source file, and insisting on real mobile interaction instead of accepting a code-only claim. This is the same instinct the product encodes in its hallucination guard and approval gates, applied by the builder to the builder's own tools.

Operational thinking. Fixes were guided by telemetry and infrastructure reality — using `context_overflow` signals and local-model routing to direct model and context changes, and asking for production-readiness checks after the NAS and Supabase work.

> **From the retrospective.** You are strongest when you steer Denizen from lived product friction, and
>
> your next step is turning that same judgment into smaller, planned, landed slices.

§C · What it signals — read against the current state

Signal	Why it lowers risk	What the current audit adds

Execution capacity	77 focused hours and a 16-hour session indicate the ability to carry a hard build to a working, tested state	Confirmed and extended: **236 commits and +266K lines in the following six weeks**, including two new product lines and the platform's first revenue
Product instinct	A "Product Thinker" profile with half of sessions on product decisions means the roadmap is steered by user friction	Confirmed — and the Discord pivot, which now carries the only paying customers, was not in any strategy document a month ago
Low oversight risk	A builder who supervises AI output and reasons from telemetry is unlikely to ship unaccountable autonomy	**Partially qualified.** The instinct is real and repeatedly evidenced — three fake-success bugs were *found and fixed by the operator*. But the guardrail layer has no tests and 64% of routes are unauthenticated, so the discipline currently lives in the operator rather than in the system
Coachable trajectory	The named growth edge — shipping in smaller planned slices — is a process refinement, not a capability gap	**Confirmed as the live constraint.** A branch 106 commits ahead of `master`, 23 uncommitted files, and production patched by hand ahead of both are the same pattern the retrospective named

The honest read for an investor. The build velocity is exceptional and independently evidenced. The risk is not capability, it is **consolidation**: production configuration is not reproducible, there is no backup, CI does not cover the server, and the safety layer is untested. Every one of those is a known, scoped, days-not-months fix — which is why they are enumerated in Appendix G rather than softened here. A second engineer's first month should be spent on that list, not on features.

Appendices

Appendix A · Command quick-reference

Task	Command
Install & run	`cd denizen && npm install && npm start`
Containerized (production)	`docker compose -f docker-compose.production.yml up -d`
~~Managed process~~	~~`pm2 start ecosystem.config.js`~~ ⚠ **broken — do not use**
Main test suite	`cd denizen && npm test`
Discord bot suite	`cd denizen/discord-bot && npm test`
CRM vendor suite	`cd denizen/crm/vendor && npm test`
All smoke tests	`npm run smoke:all`
Deploy to NAS	`py -3.12 infra/deploy-denizen-to-nas.py`
Apply DB migrations	`py infra/apply-migrations-nas.py`
Verify schema	`py infra/verify-migration.py`
Provision the mesh	`py infra/finish-all-mesh.py`
Drain mesh queue	`npm run drain:mesh`
Assign a directive	`POST /api/assign-directive`

Approve a code proposal	`POST /api/code-proposals/:id/approve`
Connect Robinhood (1×)	open `/api/market/auth/connect`
City: merge plan	`py denizen/scripts/merge-city-plan.py`
City: QA gate	`py denizen/scripts/check-city-plan.py` ⚠ **not called by merge**
City: render preview	`py denizen/scripts/render-city-preview.py --crop … --scale …`

Appendix B · Glossary

Term	Meaning
`SOUL.md`	The Markdown identity contract that becomes an agent's system prompt
`MEMORY.md`	Append-only episodic memory file, ~200-line cap, with skill/topic/outcome tags
Jelly	Denizen's in-process skill runtime (the jellyfish mascot); replaced the OpenClaw gateway
CofounderAgent	The CTO director loop that issues 2–4 commands every 15–30 s
Intelligence spine	The facts → outcomes → predictions loop
Meridian / Deal Weaver	The embedded React CRM — the business-facing surface for the agent team
Compute mesh	The Postgres-queue-backed distributed fleet with contribution credits
Directive inbox	A `DIRECT ORDER` block injected atop an agent's prompt to dominate its next decision
Hallucination guard	The grounding check that flags unsupported completion or factual claims
Protected agent	`abby`/`bouncer`/`lucy` — un-claimable and ineligible for MCP tokens
Spaces	The Discord-native walkable pixel map (`denizen/discord-map/`)
Seller slots	Time-boxed private seller channels a host guild sells to its own resellers
Home guild	A named Discord guild granted Pro with no Stripe subscription
Stamp	One placed sprite in a city fragment, anchored at a floor baseline
Fragment	One district's JSON layer set, merged into `city-plan.json`
Station agent	The desktop remote-execution surface (files, whitelisted exec, screen, input)

Appendix C · Subsystem → key file map

Subsystem	Entry file(s)
Server / routing	`server.js` (13,537 lines, no framework)
Agent brains	`src/npc-brains.js`, `src/cofounder-agent.js`
Truth + messaging	`src/world-state.js`, `src/agent-bus.js`
Capabilities	`jelly/index.js`, `src/coding-task-runner.js`, `src/task-chain.js`, `src/conflict-detector.js`
Honesty rail	`src/hallucination-guard.js`, `src/soul-reflection.js`
Intelligence spine	`src/denizen-db.js`, `src/denizen-prediction-loop.js`, `src/federal-pipeline.js`
Federal / SAM.gov	`src/denizen-sam-gov.js`, `src/denizen-crm-sam.js`
CRM	`src/denizen-crm-*.js`, `crm/vendor/` (SPA)

MCP	`src/denizen-npc-mcp-gateway.js`, `-mcp-team-gateway.js`, `-mcp-team-tools.js`, `-mcp-scopes.js`, `-mcp-oauth.js`, `-mcp-skill-jobs.js`
Discord	`discord-bot/index.js`, `discord-bot/src/modules/*`, `src/denizen-guild-plans.js`, `-guild-billing.js`, `-map-access.js`
Spaces map	`discord-map/game.js` (8,473 lines)
City / world	`src/full-world.js`, `data/city-plan.json`, `scripts/{compile-city-structure,merge-city-plan,check-city-plan,render-city-preview}.py`
Security	`security-monitor-server.js`, `src/linux-agent.js`, `src/bouncer-investigation.js`
Mesh	`src/denizen-compute-*.js`, `src/denizen-mesh-*.js`, `infra/finish-all-mesh.py`
Desktop	`desktop/main.js`, `desktop/scripts/copy-denizen-src.js`, `desktop/src/denizen-node-client.js`
Dossier	`src/dossier-system.js`, `docs/BRANDED_DOSSIER_SYSTEM.md`, `data/canonical/denizen-facts.json`

Appendix D · Verification appendix

Every headline number in this document is reproducible. Run these from the repo root and update

[the state table](#current-state-at-a-glance--v170) plus `data/canonical/denizen-facts.json` together.

```
# server.js LOC

wc -l denizen/server.js



# src module counts (top-level vs recursive)

ls denizen/src/*.js | wc -l

find denizen/src -name '*.js' | wc -l



# total JS LOC across tracked files

git ls-files '*.js' '*.mjs' | xargs wc -l | tail -1



# migrations and tables

ls denizen/supabase/migrations/*.sql | wc -l

grep -rhoE 'create table (if not exists )?[a-z_.]+' denizen/supabase/migrations | sort -u | wc -



# RLS coverage

grep -rl 'enable row level security' denizen/supabase/migrations | wc -l



# route dispatch sites and templates
```

```
grep -c "urlPath ===" denizen/server.js


# soul roster
ls -d denizen/npcs/*/ | wc -l


# tests -- the real numbers (three runners)
cd denizen            && npm test
cd denizen/discord-bot && npm test
cd denizen/crm/vendor  && npm test


# Discord command count
python -c "import json;print(json.load(open('denizen/discord-commands/commands.json'))['count'])


# env vars (remember: process.env grep undercounts by ~63)
grep -rhoE 'process\.env\.[A-Z0-9_]+' denizen --include='*.js' | sort -u | wc -l


# city plan
python -c "import json;d=json.load(open('denizen/data/city-plan.json'));print(d['worldSize'],{k:
py denizen/scripts/check-city-plan.py          # expect 56 defects today


# asset count
find denizen/assets/limezu -name '*.png' | wc -l


# docs
ls denizen/docs/*.md | wc -l


# what changed since this document was last verified
git log --oneline d25bc7a2..HEAD
git diff --shortstat d25bc7a2..HEAD
```

Appendix E · Delta from v1.6.0

Pinned commit moved `ae3ddf9` (2026-06-13) → `d25bc7a2` (2026-07-24): **236 commits, +266,389 / −10,739 lines.**

Corrected numbers

Claim in v1.6.0	Actual at `d25bc7a2`	Δ
646/646 tests passing	**1,432 pass / 7 fail** of 1,439 (1,581 across 3 runners)	+2.4× count; **not green**
95 test files	**192** in the main suite (219 total)	+102%
9,562-line server	**13,537**	+42%
210 runtime modules	**334 top-level / 374 recursive** (18 of the 30 largest are browser code)	+59%
31 migrations	**51** (47 applied in production)	+20
72+ tables	**77** + 13 external	—
16 agents	**18 in the roster** (20 `SOUL.md` files, 2 orphans)	+2
Claude cloud for 7 agents, Grok cloud for 2	**No agent uses a cloud provider as its declared backend** — all `lmstudio`/`lmstudio-fast`, plus 2 CLI bridges	corrected
Think cadence 45–75 s	**120–200 s**	corrected
Hallucination guard enforces	**Advisory; caller OFF by default; verifier is dead code**	corrected
Daily SOUL reflection runs	**`reflectOnDay` has zero callers**	corrected
Skills (~11 named)	**29** (23 native + 6 vendor)	enumerated
Task chains durable	**In-memory only; lost on restart; gates fail open**	corrected
3-node compute mesh	**Unbounded model; one active general worker**	claim withdrawn
4 shipped clients	**3 shipped + 1 partial**; Android is not a client	−1
106 docs	**93** in `denizen/docs`	−13
3 MCP tools	**38 distinct**	+12×
Desktop v0.1.4	**v0.1.6** — and unbuildable since 2026-06-30	—
4 WebSocket channels	**2 WS + 2 SSE**	corrected
~82.5% prediction accuracy	**withdrawn pending re-derivation**	—
Qwen2.5-14B / dolphin3.0-8b tiers	**8B main / 14B heavy / 4B fast**, restructured 2026-07-19	corrected
Gemini listed as a provider	**No Gemini provider exists**	removed
CI on Ubuntu + Windows × Node 20 + 22	**No server CI exists**	corrected
`pm2 start ecosystem.config.js`	**Broken file — raises TypeError**	corrected
"Org-scoped RLS everywhere — Met"	**37 of 77 tables**	corrected

New sections

Part I §11 (Discord product), §12 (city & world system); Part II §4 Discord requirements, personas

persona 4, stories US-11–13; the SAM.gov subsection in §10; the MCP expansion in §10; Appendix D (verification) and Appendix G (defect register).

Claims that survived unchanged

No Express (raw `http` + `ws`); the soul-file pattern and memory tag conventions; the decision JSON contract and schema-in-user-message finding; the two decision loops; the coding-agent worktree flow and its guardrails; task chains, conflict detection and Abby arbitration; proposal queues and daily budgets; the 10-category threat catalog and Bouncer's investigate-not-just-alert design; the two-Kali distinction; `placeTrade()` does not exist; the layer model's intent; zero-config first boot; the Part IV retrospective.

Appendix E2 · Delta from v1.7.0

Pinned commit moved `d25bc7a2` (2026-07-24) → `1cab7977` (2026-09-16): **119 commits, +536,492 / −7,983 lines across 469 files.**

Headline scale (re-measured)

Claim in v1.7.0	Actual at `1cab7977`	Δ
13,537-line server	**13,929**	+392
334 top-level `src/` modules	**394** `.js` under `src/`	+60
246,487 LOC / 846 JS files	**257,563 LOC / 886** tracked JS	+~11k LOC
51 migrations	**52**	+1
93 docs	**97**	+4
Customer objects unnamed	**Floor · Workbench · Buzz · Seat an AI · Studio Board**	new

New product surfaces (missing from v1.6 PDF and under-documented in v1.7)

Surface	What shipped	Evidence
Buzz capture	Ring UI, context-drop to Abby, desktop overlay, mobile gesture/mic fixes	`src/buzz-capture.js`, PR #72/#73
Workbench rail	In-office orchestrator chip/rail	`src/work-rail.js`
Seat an AI	Provider seating catalog + API	`src/seat-ai.js`
Studio Board	Phone app + Monday parity + MCP + CRM agency link	PRs #60–#71
CRM member ACL / team viewer	Operator records scoped; AI/MCP org-wide; team activity	PR #67
Living office	Furniture use, unstuck walk map, no teleport-after-wander	PRs #56–#58
Mobile HQ / landing polish	Portrait map box, Buzz landing, phone login	PRs #55, landing commits

Still true from v1.7 honesty rail

Prediction accuracy remains withdrawn; desktop Electron was unbuildable in July; VisionClaw is still

not counted as a Denizen client; do not claim "CI on every push" without checking current workflows;

re-run Appendix D before quoting suite pass counts externally.

Appendix F · Positioning language

> **One-liner.** Denizen is the operating layer for your AI company — Floor, Workbench, and Buzz —
>
> a pixel-art office where every agent is a real, observable, accountable LLM teammate.

Audience	Line
Category	A real-time visual observability layer for AI agents
For founders	Run an AI company you can actually watch — Floor for the team, Workbench for the work, Buzz to drop context
For engineers	See the shape of your agent fleet's work, not a wall of logs — with approval gates on everything that mutates code or identity
For Discord operators	Give your server a place your members can walk into, and moderation that actually holds — priced per server
The wedge	Legible by default, safe by construction, portable to any vertical by editing a Markdown file

⚠ **Do not use in external material until resolved:** any specific prediction-accuracy percentage; "all

tests passing"; "3-node mesh"; "4 shipped clients"; "org-scoped RLS everywhere"; "CI on every push";

Android/VisionClaw as a Denizen client; AI Prospector as a working feature; Marketing campaign send as

functional.

Appendix G · Open defect register

Ranked by blast radius. Every item was verified during the 2026-07-25 survey and cites its location.

Items 1–3 warrant attention before any external distribution of this document.

#	Item	Location	Severity
1	**Live client data untracked but not gitignored** — a repair report containing real deal, company and contact rows sits in `infra/`, one `git add -A` from being committed. Its sibling secrets *are* ignored	`infra/repair-crm-cwr43-report.json`	**Critical — data exposure**
2	**Bouncer auth fails open**, and two WS triggers reach WSL command execution with no auth at all	`src/bouncer-investigation.js:275`, `server.js:3474`, `:12733`, `:12812`	**Critical — RCE surface**
3	**No database backup exists**	repo-wide	**Critical — data loss**
4	**64% of route dispatch sites unauthenticated**, including `soul-proposal/apply`, `assign-directive`, `grant-permission`, all market writes, all telemetry	`server.js` (237 of 372 sites)	High
5	**Cross-tenant leak** — the federal pipeline summary queries `rfps` with `filters: {}` under a service-role key	`src/federal-pipeline.js:192`	High
6	**Production config is not reproducible** — the deploy script writes `.env`, compose reads	`infra/deploy-denizen-to-nas.py:355,401`	High

	`.env.production`		
7	**The intelligence spine is not closed in production** — 4 migrations hard-skipped	`infra/apply-migrations-nas.py:24-29`	High
8	**Desktop unbuildable since 2026-06-30** — 2 transitive requires missing from the copy whitelist. Also blocks shipping the mesh fake-success fix	`desktop/scripts/copy-denizen-src.js:10-19`	High
9	**7 failing tests / 6 files**, suite exits 1 — including 2 nominally security-relevant avatar-guard assertions	see Part I §17	High
10	**The hallucination guard is disabled by default and untested** — advisory only, caller gated on `DENIZEN_REAL_WORK_MODE`, which `prompt-tuning.js:86` hardcodes false on both ternary branches; the model verifier is dead code; the flag event has no consumer	`src/hallucination-guard.js`, `npc-brains.js:5006`, `prompt-tuning.js:86`	**High — this is the stated mitigation for the project's headline risk**
11	**`claude-haiku-3-5-20241022` is retired and is the hard default for `DENIZEN_PUBLIC_DEMO_MODEL`** — requests 404; propagated to the NAS by two provisioning scripts. The ID is also malformed	`prompt-tuning.js:43`, `infra/provision-nas-server.py:141`, `infra/patch-denizen-nas-env.py:33`	High
12	**Zero tests on the money paths** (10 credits/billing modules) and **the safety layer** (6 guardrail modules)	—	High
13	**No server CI** — only a desktop-release workflow	`.github/workflows/`	High
14	**Task chains have no persistence** — in-memory `Map`s, lost on every restart; content gates **fail open**; a second chain per NPC silently overwrites the first	`src/task-chain.js:10-11`, `src/content-gate.js:63`	High
15	**`reflectOnDay` has zero callers** — the SOUL reflection generator never runs, so the identity-evolution loop the design depends on is inert	`npc-brains.js:6736`	Medium
16	**Think-cadence fallback disagrees with config 3×** — a failed `/api/office-config` fetch silently slows the whole office from 120 s to 360 s	`agent-office-manager.js:318-321` vs `prompt-tuning.js:49-50`	Medium
17	**`route-registry.js` and `routes/intelligence-routes.js` are entirely dead** — imported, never called; three docs attribute live endpoints to them	`server.js:171`	Medium
18	**Agent bus is single-process only** — an in-process EventEmitter with no Redis or DB fan-out, so multi-instance deploys silo NPC↔NPC messaging	`src/agent-bus.js:23-29`	Medium
19	**Two orphan soul directories** — `npcs/marcus/` and `npcs/sarah/` are never loaded and corrupt any `ls`-based count	`src/npc-roster.js:8-27`	Low
20	**`observe` advertised but unimplemented; `idle` unhandled** — `observe` is in the think prompt but not the schema or dispatcher; `idle` is in the schema but falls to `default:` and behaves as `work`	`npc-brains.js:3347`, `:7103`, `agent-office-manager.js:2682`	Low
21	**`ROLE_BLURBS` is missing `claude`** — 17 keys for 18 agents, so generic-pack orgs get a fallback blurb	`src/generic-soul-pack.js`	Low
22	**`DISCORD_SLASH_TRIM` unset** — the next sync silently drops staff commands instead of game commands	`denizen/.env`, `command-registry.js:30`	Medium

23	**Discord music reports false success** — no voice encryption dep installed	`discord-bot/src/modules/music/index.js:204-228`	Medium
24	**Non-commercial-labeled art in a paid SKU**	`discord-map/game.js:852`	Medium — legal
25	**No container log rotation**; `llm-telemetry.jsonl` at 54 MB	`docker-compose*.yml`, `data/`	Medium
26	**Migration chain defects** — a version collision silently skips one migration; two never record a version; a duplicate index has conflicting uniqueness	`denizen/supabase/migrations/`	Medium
27	**`prediction-summary` always returns `topForecasts: []`** — the live handler calls `getPredictionSummary(orgId, db)` with no options, so the `includeTopForecasts` branch never runs	`server.js:7730` vs `denizen-prediction-loop.js:147`	Medium
28	**`deploy-full-world-nas.py` ships assets but not the city** — `FILES` omits `city-plan.json` and `city-specs/`	`infra/deploy-full-world-nas.py:31-41`	Medium
29	**City QA unenforced** — merge never calls the gate; 56 defects currently outstanding	`scripts/merge-city-plan.py`	Medium
30	**`_asset-catalog.json` has no generator on disk** — reproducibility gap, already stale by 8 files	`denizen/assets/limezu/`	Medium
31	**`bbox` never updated on relocate** — `d10` has 0 of 51 stamps inside its declared box	`scripts/relocate-district.py:31-56`	Low
32	**13 CRM routes missing from the `/api/crm/health` deploy diagnostic**	`src/denizen-crm-routes.js`	Low
33	**MCP batch JSON-RPC drops all but the first message**; `clear_directive` unscoped; SQL scope fallback omits `compute`; 1-click OAuth mints `full`	`src/denizen-npc-mcp-gateway.js:594-599`, `-mcp-scopes.js`, `-mcp-oauth.js:294`	Low
34	**MCP memory is one-way in practice** — in-world prompts don't see MCP writes until restart	`src/npc-brains.js:1590-1620`	Low
35	**AI Prospector and Marketing send are UI-only but documented as real**	`crm/vendor/src/.../AIProspector.tsx:16`, `MarketingTab.tsx:73-80`	Low — accuracy
36	**`generate-city-endpoint.test.js` hangs 30 s every run** — `after()` doesn't reap	`denizen/tests/`	Low
37	**AtLogOn scheduled tasks die on logoff**; Kali agent has no systemd unit; **LM Studio is hand-started**	`infra/*.ps1`, `infra/README.md:209-215`	Low
38	**Dead config and code** — `GEMINI_API_KEY`, `DENIZEN_HOST_ON_NAS`, `LAVALINK_AUTO`, `ecosystem.config.js`, `desktop/compute-job-runner.js`, the `?debug=city` overlay	various	Low

Stale companion docs

The survey found significant drift in the following, listed so they can be corrected or retired:

DATABASE_SCHEMA.md (31 undocumented tables, 7 phantom tables),

CURRENT_IMPLEMENTATION_STATUS.md (omits every surface since

2026-07-16; the Bouncer feature is entirely absent),

NPC_MCP_GATEWAY.md (titled v1.3, documents 16 of 38 tools, omits the `compute`

scope), MCP_POWER_GAP_PLAN.md (its "current state" paragraph contradicts its own ✅ checklist), COMPUTE_MESH.md (3 job types of 9, 8 endpoints of 21), TESTING.md (never mentions `npm test`), OPS_LIVE_STACK.md (names a branch 140 commits behind), PRODUCTION_DEPLOY.md (port 8080; instructs setting a dead key), FEDERAL_PIPELINE.md (never mentions SAM.gov), DISCORD_PAID_TIER.md (documents a `,upgrade` command that does not exist), DISCORD_BOT_AND_SPACES.md (map gating order is now inverted), FULL_WORLD.md and SPRITE_PIPELINE.md (asset counts and world size superseded; calls a shipped feature "missing"), `RUNTIME_DATA.md` (omits ~26 present runtime files), `MONOREPO.md` (omits `discord pixel game/`, says sixteen NPCs), and `data/city-specs/city-plan.md` (every coordinate superseded).

Additionally, from the roster/harness survey:

ACTIONS.md documents 13 JS actions and 12 `[ACTION:*]` tags but **omits all 13 think-loop JSON actions**, which are the primary autonomy vocabulary (the union across all dispatchers is 43 distinct names); it also names a `callMeeting` method on `AgentActions` that does not exist, points at a `_handleNpcDecision` that is actually `_executeNpcDecision`, and its line counts are off by 3×.

SOUL_REFLECTION.md describes a daily loop that is not wired.

AGENT_BUS.md claims 11 test behaviors (there are 9), describes a bare envelope response (it is nested), and omits both the think-wake coupling on every message and the 600-char truncation on the HTTP path. HARNESS_EVALUATION.md cites "70 skills" — that is the upstream OpenClaw count, not Denizen's 29. WORKSPACE_ARCHITECTURE.md calls `hallucination-guard.js` a "guardrail," which is misleading given nothing acts on its verdict.

DENIZEN_HARNESS.md documents only the production `main` model, not the dev split.

"Sixteen agents" also appears in shipped prose, not just docs — `src/generic-soul-pack.js:55` injects "sixteen teammates" into every generic-pack soul prompt, and `src/demo-scene.js:320` renders "Sixteen agents, one office." Those are user-visible and should be corrected with the docs.

DENIZEN · v1.8.0 · source of record · headlines at `1cab7977` (2026-09-16) · deep survey `d25bc7a2` (2026-07-25)

Render via the `pdf-dossier` skill against `data/canonical/denizen-facts.json`.

Approval required before any external send — see BRANDED_DOSSIER_SYSTEM.md §5.